

Theory of Computation (BCSE0404X(TOC))

Module: 4

**Parser and Intermediate
Representation**

**Course Details
(B.Tech CSE 4th Sem)**



Vivek Kumar Sharma
Assistant Professor
CSE



Brief Introduction of Faculty

Vivek Kumar Sharma

Designation: Assistant Professor CSE Department

NIET Grater Noida

Qualifications:

- B.Tech (CSE) in 2010
- M.Tech (CSE) in 2013

Teaching Experinces: 10 year +

Research Publications:

Particulars	Journals (UGC)	Conference(IEEE)
International	04	02
National	01	00

Evaluation Scheme

**NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY, GREATER NOIDA, GAUTAM BUDDH NAGAR
(AN AUTONOMOUS INSTITUTE)**

**Bachelor of Technology
Information Technology**

**Evaluation Scheme
SEMESTER-IV**

Sl. No.	Subject Codes	Subject	Types of Subjects	Periods			Evaluation Schemes				End Semester		Total	Credit
				L	T	P	CT	TA	TOTAL	PS	TE	PE		
1	BASCC0401	Employability Skill Development - II	Mandatory	2	0	0	60	40	100				100	2
2	BCSE0402	Database Management Systems	Mandatory	3	0	0	30	20	50		100		150	3
3	BCSE0401	Data Structures and Algorithms-II	Mandatory	3	0	0	30	20	50		100		150	3
4	BCSE0404X	Theory of Computation	Mandatory	3	0	0	30	20	50		100		150	3
5		Department Elective - I	Departmental Elective	3	0	0	30	20	50		100		150	3
6	BAS0403N	Statistics and Probability	Mandatory	3	1	0	30	20	50		100		150	4
7	BCSE0452Z	Database Management Systems Lab	Mandatory	0	0	4				50		50	100	2
8	BCSE0451	Data Structures and Algorithms-II Lab	Mandatory	0	0	2				25		25	50	1
9	BCSE0455	Web Technologies	Mandatory	0	0	6				50		100	150	3
10	BCSE0459	Mini Project	Mandatory	0	0	2				50			50	1
11	BCSCC0452	Problem Solving Approaches	Mandatory	0	0	2				50			50	1
12	BNC0402Y/ BNC0401Y	Environmental Science / Artificial Intelligence and Cyber Ethics	Compulsory Audit	2	0	0	30	20	50				50	NA
		*Massive Open Online Courses (For B.Tech. Hons. Degree)	*MOOCs											
		Applied English	VAC	1	0	0								
		TOTAL		20	1	16			350	225	500	175	1250	26

Syllabus

Course Contents / Syllabus		
Module 1	Introduction to Finite Automata and Compiler	10 hours
Introduction: Role of Automata and Formal languages, Alphabet, String, Grammar, Language, Chomsky Hierarchy of languages. Introduction to Finite State Machine: Deterministic Finite Automaton (DFA) and Non-Deterministic Finite Automaton (NFA), NFA with ϵ -Transition, Equivalence of NFA and DFA, Introduction to Compiler: Translators, Language Processing System, Phases and passes of compilation Use Case 1 : Role of Finite Automata in NLP and Speech Recognition.		
Module 2	Regular Expression and Tokenization	9 hours
Regular Expression: Regular Expression, Regular Sets, Properties of Regular Expression, Finite Automata and Regular Expression, Arden's Theorem, Regular Grammars-Right Linear and Left Linear grammars. Lexical Analyzer: Role of lexical Analyzer, Specifications and Recognition of tokens, Lex,		
Module 3	Context free grammar and Push Down Automata	09 hours
Context Free Grammar (CFG): Definition and Language, Derivations, Parse Trees and Ambiguity, Simplification of CFG, Push Down Automata: Definition of the Pushdown Automata, Languages of PDA		
Module 4	Parser and Intermediate Representation	10 hours
Parser: Role of parser, Top down Parsing-LL (1) parser, Bottom up parsing- shift reduce parser and LR (0), SLR parser, Introduction to Syntax directed Translation, Intermediate-Code Generation: Three-Address Code- Quadruples, Triples, Indirect triples Use Case-2: Role of CFG and parsing in Voice Assistant		
Module 5	Turing machine and Optimization	10 hours
Turing Machine: Basic Concept of Turing Machine, Variants of Turing Machine, Universal Turing Machine Code optimization and generation: Basic Block, Flow graph, DAG, Optimization Techniques		
Total Lecture Hours		48 hours
Textbook:		
S.No	Book Title	
1	K.L.P. Mishra, and N. Chandrasekharan , Theory of Computer Science-Automata, Languages and Computation	
2	Alfred V. Aho, Jeffrey D Ullman , Compilers: Principles, Techniques and Tools	

Branch Wise Applications

- It can compute man-made problems as well as natural phenomena.
- Automata theory has a lot of applications in real life as well, such that: Lambda calculus, Combinatory logic, Markov algorithm, and Register, Natural Language Processing ,Compiler Design and Lexical analysis and Semantic analysis. Also, the concept of Formal languages and automata theory can overlap with other subjects as well.

Course Objectives

- The primary objective of this course is to provide a foundational understanding of Automata Theory and its role in the language processing systems, also explores their application in fields like Natural Language Processing (NLP), speech recognition.

Course Outcome

Theory of Computation (BCSE0404X)

Course Outcome (CO)		Bloom's Knowledge Level (KL)
After completion of the course, the student will be able to:		
CO 1	Identify the fundamental concepts of automata theory, formal languages and compiler components.	K2
CO 2	Understand the role of finite automata, regular expressions, and grammar rules in language processing.	K2
CO 3	Demonstrate context-free grammars, pushdown automata, and syntax-directed translation to construct intermediate code for language processors.	K3
CO 4	Analyze various parsing strategies, code translation methods, and intermediate representations in compiler phases.	K4
CO 5	Analyze the functioning of Turing machine models and optimization techniques in code generation for performance improvement.	K4

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

Program Outcomes (POs)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

Program Outcomes (POs)

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

CO-PO Correlation Matrix

CO-PO Mapping	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	2	2	-	-	-	-	-	-	2
CO2	3	3	3	2	1	-	-	-	-	-	2
CO3	3	3	3	3	1	-	-	-	-	-	2
CO4	3	3	3	3	1	-	-	-	-	-	2
CO5	3	3	3	3	1	-	-	-	-	-	2
Average	3	2.8	2.8	2.6	1	-	-	-	-	-	2

Program Specific Outcomes

On successful completion of graduation degree, the Information Technology graduates will be able to:

PSO1	Work as a software engineer in different dimensions of information technology to provide solutions to real-world and industrial problems in an eco-friendly manner
PSO2	Apply knowledge of core subjects with multi-disciplinary and modern computing techniques to solve complex IT problems.
PSO3	Establish a successful and innovative career by practicing lifelong learning and communicating in a team or as an individual to manage projects with ethical concerns.

CO-PSO Correlation Matrix

CO-PSO Mapping	PSO1	PSO2	PSO3
CO1	2	3	1
CO2	2	3	1
CO3	3	3	2
CO4	3	3	2
CO5	3	2	2
Average	2.6	2.8	1.6

End Semester Question Paper Template

Printed Page:	Subject Code:
	Roll. No:
NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA	
(An Autonomous Institute Affiliated to AKTU, Lucknow)	
B.Tech	
SEM: - THEORY EXAMINATION (2025 - 2026)	
Subject:	
Time: 3 Hours	Max. Marks: 100
General Instructions: IMP: Verify that you have received the question paper with the correct course, code, branch etc. 1. This Question paper comprises of three Sections -A, B, & C. It consists of Multiple-Choice Questions (MCQ's) & Subjective type questions. 2. Maximum marks for each question are indicated on right -hand side of each question. 3. Illustrate your answers with neat sketches wherever necessary. 4. Assume suitable data if necessary. 5. Preferably, write the answers in sequential order. 6. No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.	

End Semester Question Paper Templates

<u>SECTION-A</u>		20
1. Attempt all parts:-		
1-a.		1
1-b.		1
1-c.		1
1-d.		1
1-e.		1
1-f.		1
1-g.		1
1-h.		1
1-i.		1
1-j.		1
1-f.		1
2 Attempt all parts:-		
2.a.		2
2.b.		2
2.c.		2
2.d.		2
2.e.		2

End Semester Question Paper Templates

SECTION-B		30
3. Attempt all parts: -		
3.a. Answer any one of the following: -		
3.a.(i)		6
3.a.(ii)		6
3.b. Answer any one of the following: -		
3.b.(i)		6
3.b.(ii)		6
3.c. Answer any one of the following: -		
3.c.(i)		6
3.c.(ii)		6
3.d. Answer any one of the following: -		
3.d.(i)		6
3.d.(ii)		6
3.e. Answer any one of the following: -		
3.e.(i)		6
3.e.(ii)		6

End Semester Question Paper Templates

<u>SECTION-C</u>		50
4. Answer any <u>one</u> of the following: -		
4-a.		10
4-b.		10
5. Answer any <u>one</u> of the following: -		
5-a.		10
5-b.		10
6. Answer any <u>one</u> of the following: -		
6-a.		10
6-b.		10
7. Answer any <u>one</u> of the following: -		
7-a.		10
7-b.		10
8. Answer any <u>one</u> of the following: -		
8-a.		10
8-b.		10

Program Educational Objectives

PEO'S of the B. Tech. in Information Technology are:

The graduates will be:

PEO1: To apply sound knowledge combining technical expertise, and problem-solving skills to fulfill the needs of the IT industry and stay updated with the latest emerging trends.

PEO2: To provide sustainable solutions for real-life problems by designing innovative and interdisciplinary systems using the latest digital technologies for serving academia, industry, and society.

PEO3: To reflect professional attitude, social ethics, communication, teamwork, and leadership skills in research and product development to face global challenges.

PEO4: To pursue higher studies and develop their career successfully in the software industry and R&D sectors with a lifelong learning frame of mind.

.

Unit Objective

- To understand the definition of Context Free Grammar (CFG) and the types of languages generated by CFG.
- To explain the concept of derivations in CFG and analyze how strings are generated step by step.
- To study parse trees and identify ambiguity in grammars through multiple derivations.
- To learn the techniques for simplification of CFG by removing useless symbols, null productions, and unit productions.
- To understand the definition and structure of Pushdown Automata (PDA) as a computational model.
- To analyze the relationship between Pushdown Automata and Context Free Languages, and identify the languages accepted by PDA.

Topic Objective

Topic	Objectives
Context Free Grammar (CFG): Definition and Language	To understand the formal definition of Context Free Grammar and identify the class of languages generated by CFG.
Derivations in CFG	To learn leftmost and rightmost derivations and analyze the step-by-step generation of strings in CFG.
Parse Trees and Ambiguity	To construct parse trees and recognize ambiguous grammars through multiple parse trees or derivations.
Simplification of CFG	To understand and apply methods for simplifying CFG by eliminating useless symbols, null productions, and unit productions.
Pushdown Automata (PDA): Definition	To understand the structure, components, and working of Pushdown Automata as an abstract machine.
Languages of Pushdown Automata	To analyze the types of languages accepted by PDA and establish the equivalence between PDA and Context Free Languages.

Prerequisite:

- Basic knowledge of formal languages and automata concepts such as alphabets, strings, and languages.
- Understanding of regular languages, regular expressions, and finite automata (DFA/NFA).
- Familiarity with grammar basics, including terminals, non-terminals, production rules, and derivations.
- Basic understanding of set theory and mathematical notation used in language definitions.
- Awareness of language recognition models and their role in classifying languages.

Automata
Theory

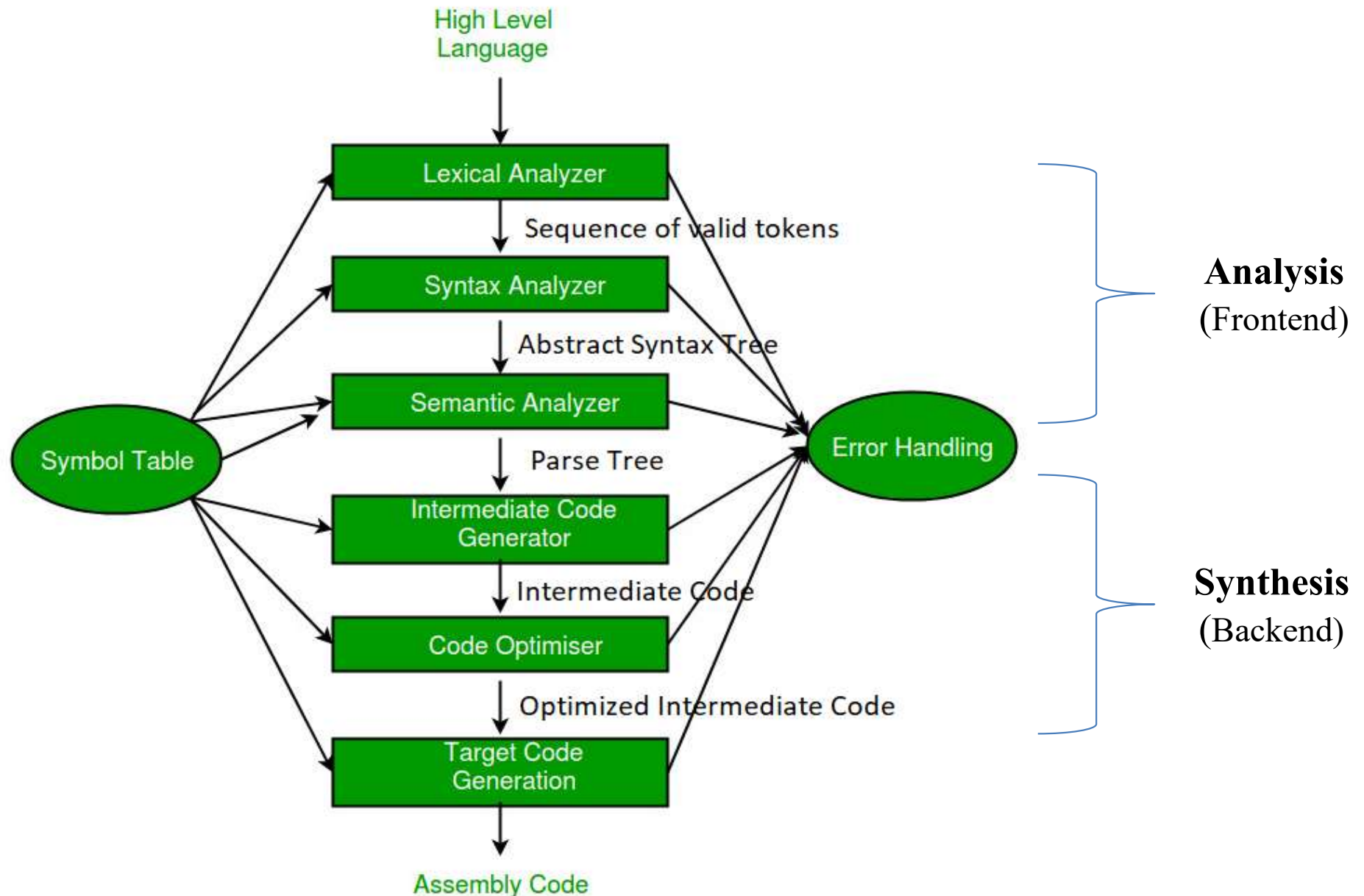
Context
Free
Languages

Data
Structure

Logic or
Algebra

Simple
Graph
Algorithms

Computer
Architecture



- Course Objective
- Course Outcome
- CO-PO and PSO Mapping
- Prerequisite and Recap
- Objective of Topics
- Basic Parsing Techniques: Parsers
- Shift reduce parsing
- Top down parsing,
- Predictive parsers Automatic Construction of efficient Parsers: LR parsers
- The canonical Collection of LR(0) items,

- Constructing SLR parsing tables
- Implementation of LR parsing tables.
- Faculty Video Links, Youtube & NPTEL Video Links and Online Courses Details
- Daily Quiz
- Weekly Assignments
- MCQ
- Expected Question for University Exam/old question paper
- References

Unit Objective

Topic	Objective
Basic Parsing Techniques	To learn about different parsing techniques.
Top Down Parsing	Understand the logic behind top down parsing technique i.e. Predictive parsing technique.
Bottom Up Parsing	To know about different bottom-up parsing techniques i.e. shift-reduce parsing techniques, operator precedence parsing etc.
LR Parsing	To understand the four different types of LR parsing techniques

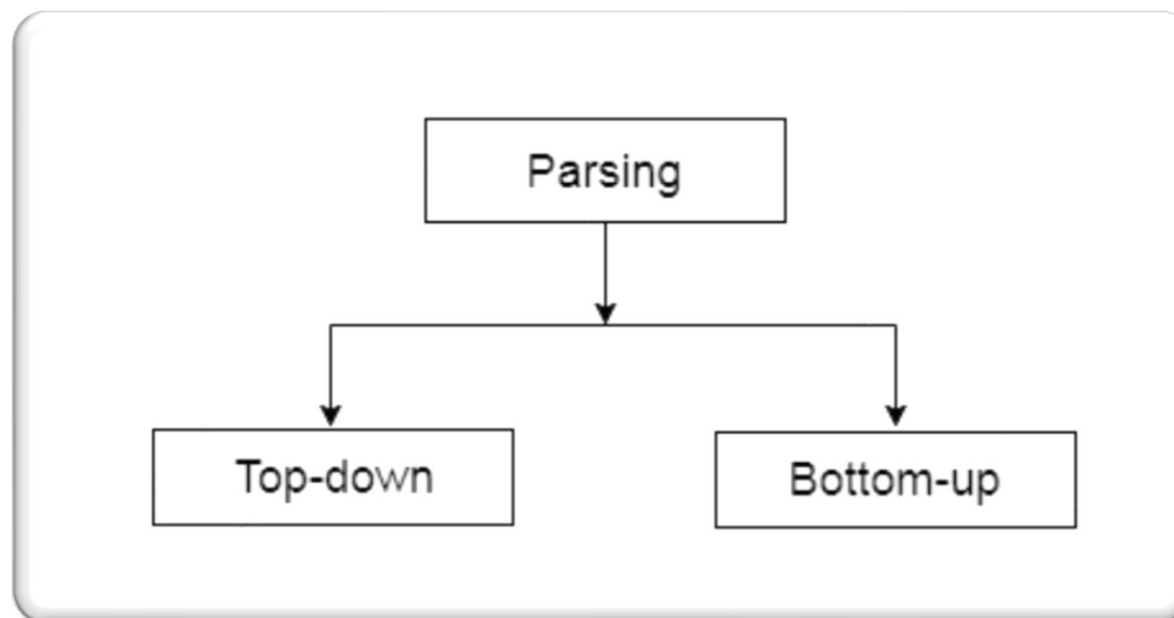
Topic Objective(CO1)

1. Introduce students to the concepts of Top Down and Bottom Up Parser .

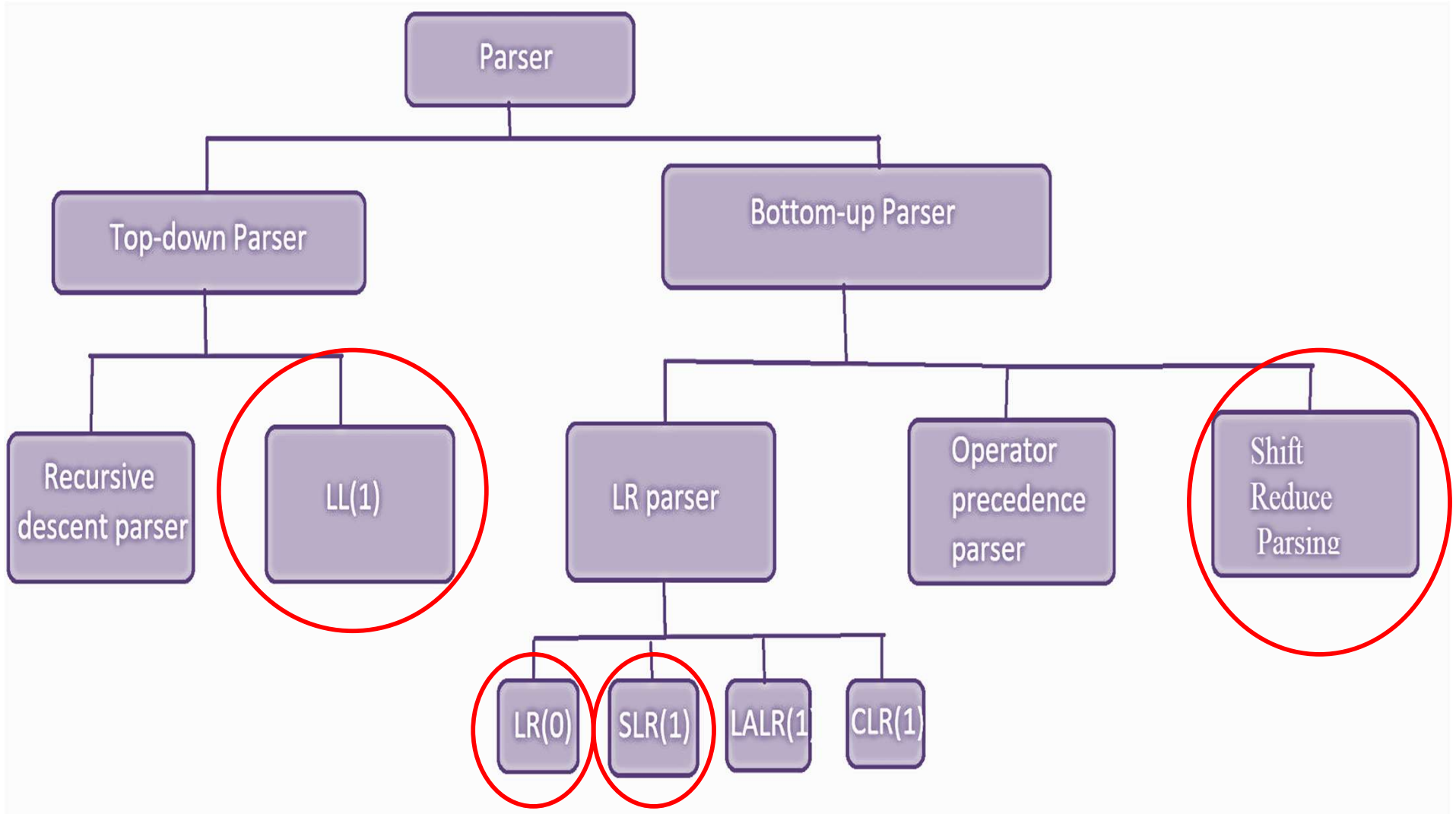
Basic Parsing Techniques(CO2)

Parsing (also known as **syntax analysis**) can be defined as a process of analyzing a text which contains a sequence of tokens, to determine its grammatical structure with respect to a given grammar.

Depending upon how the parse tree is built, parsing techniques are classified into two general categories



Basic Parsing Techniques(CO2)



Basic Parsing Techniques

Since a parse tree is the graphical representation derivation.

Top-Down parsing is equivalent to left-most derivation

Bottom-Up parsing is equivalent to reverse of right most derivation

Top-Down Parse(CO2)

It is the parser which generate parser for the given input string with the help of grammar production by expanding the non-terminals i.e. it starts from the start symbol and ends on the terminals.

Top Down parser uses left most derivation.

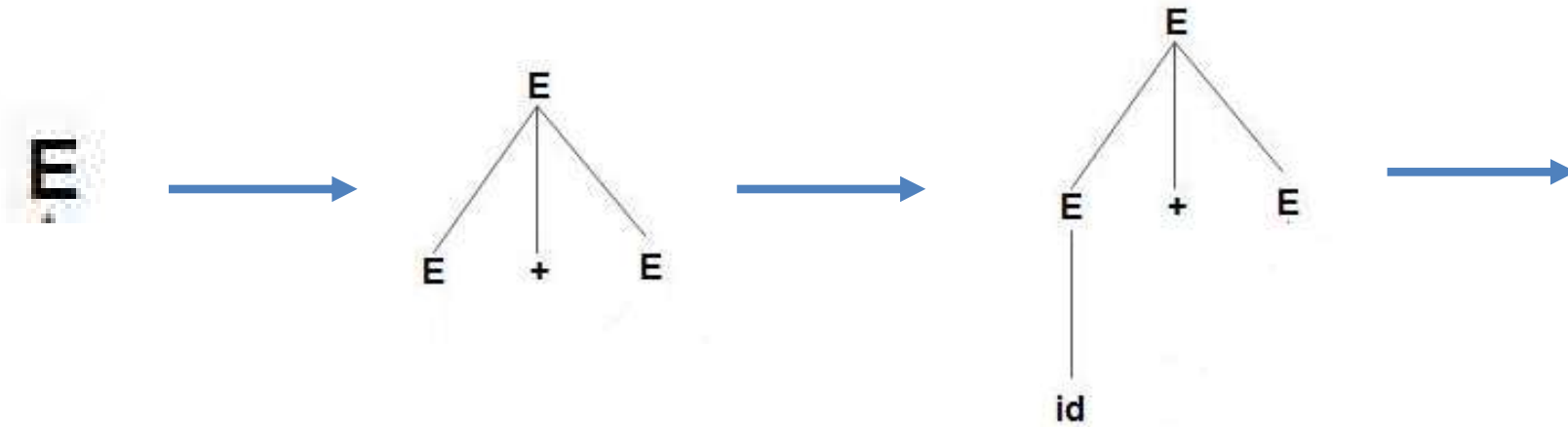
Top down parser can be constructed for the grammar if it is free from ambiguity and left recursion.

Top-Down Parse Example

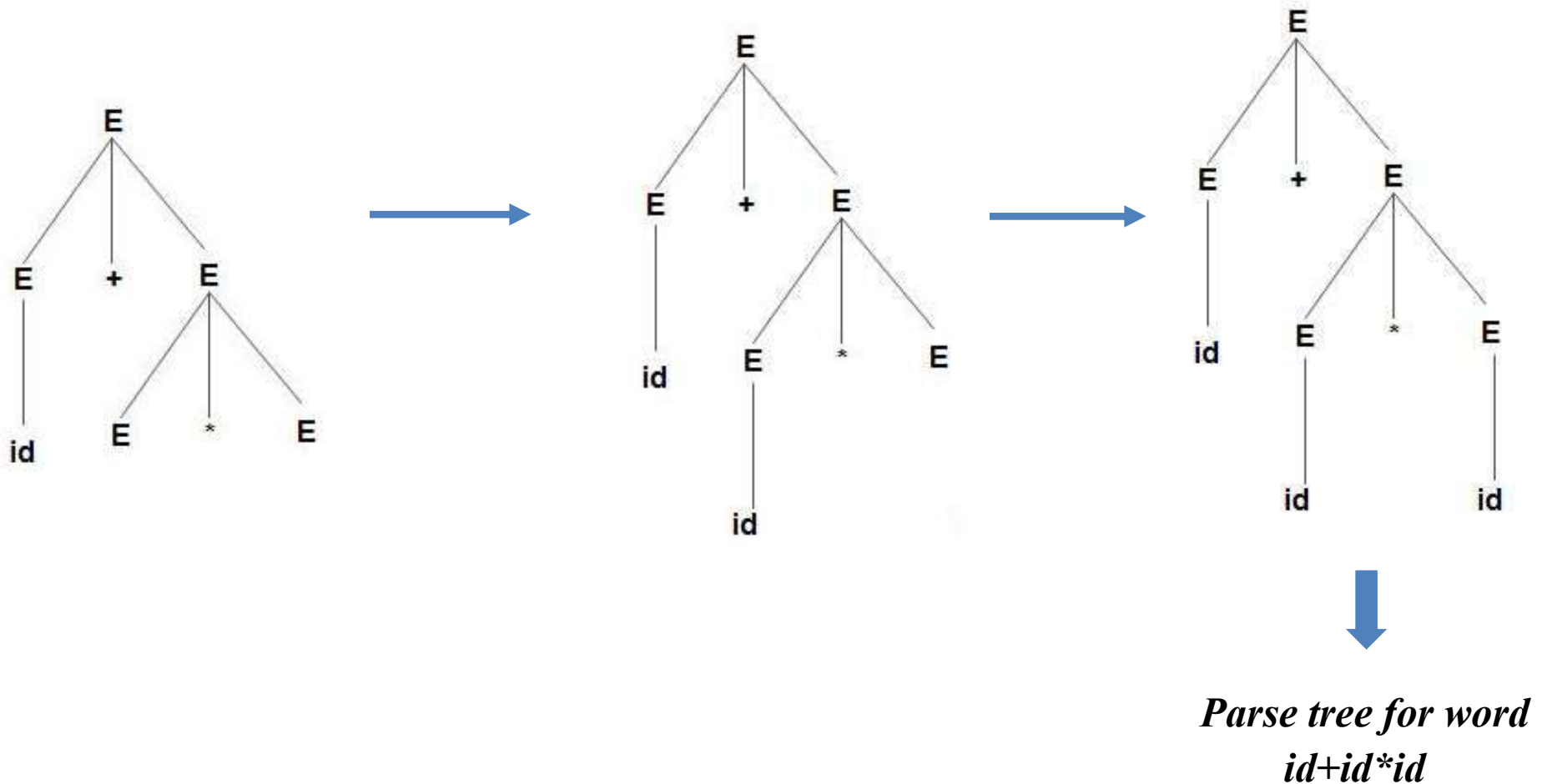
Construct parse tree for

$$E \rightarrow E + E \mid E * E \mid id$$

using Top Down Parsing Technique



Top-Down Parse Example



Bottom-Up Parse(CO2)

Bottom-up parsing starts from the leaf nodes of a tree and works in upward direction till it reaches the root node.

Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol.

Bottom up parser can be constructed for the grammar if it is free from ambiguity.

Bottom-Up Parse Example

Example: Input string is id+id*id and production rules are as given below:

$S \rightarrow E$

$E \rightarrow E + T$

$E \rightarrow E * T$

$E \rightarrow T$

$T \rightarrow \text{id}$

Perform bottom-up parsing technique.

Solution: Starting with input string id+id*id

id + id * id

T + id * id

E + id * id

E + T * id

E * id

E * T

E

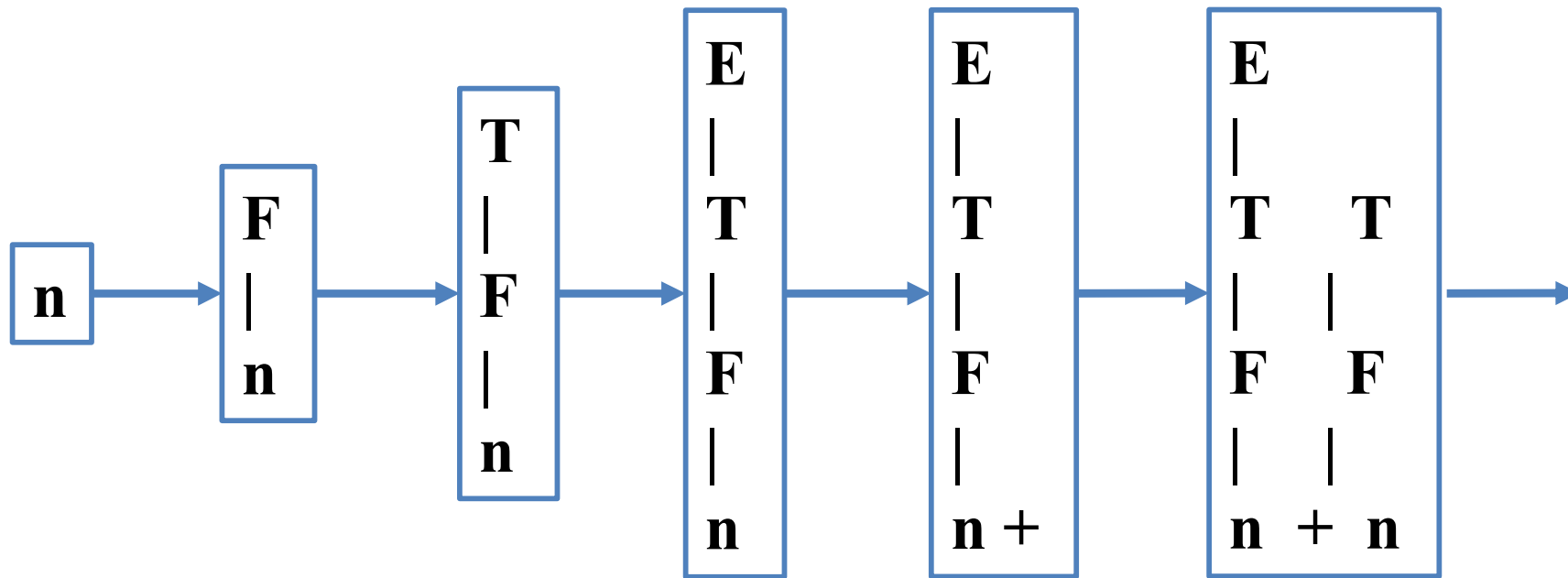
S

Bottom-Up Parse Example

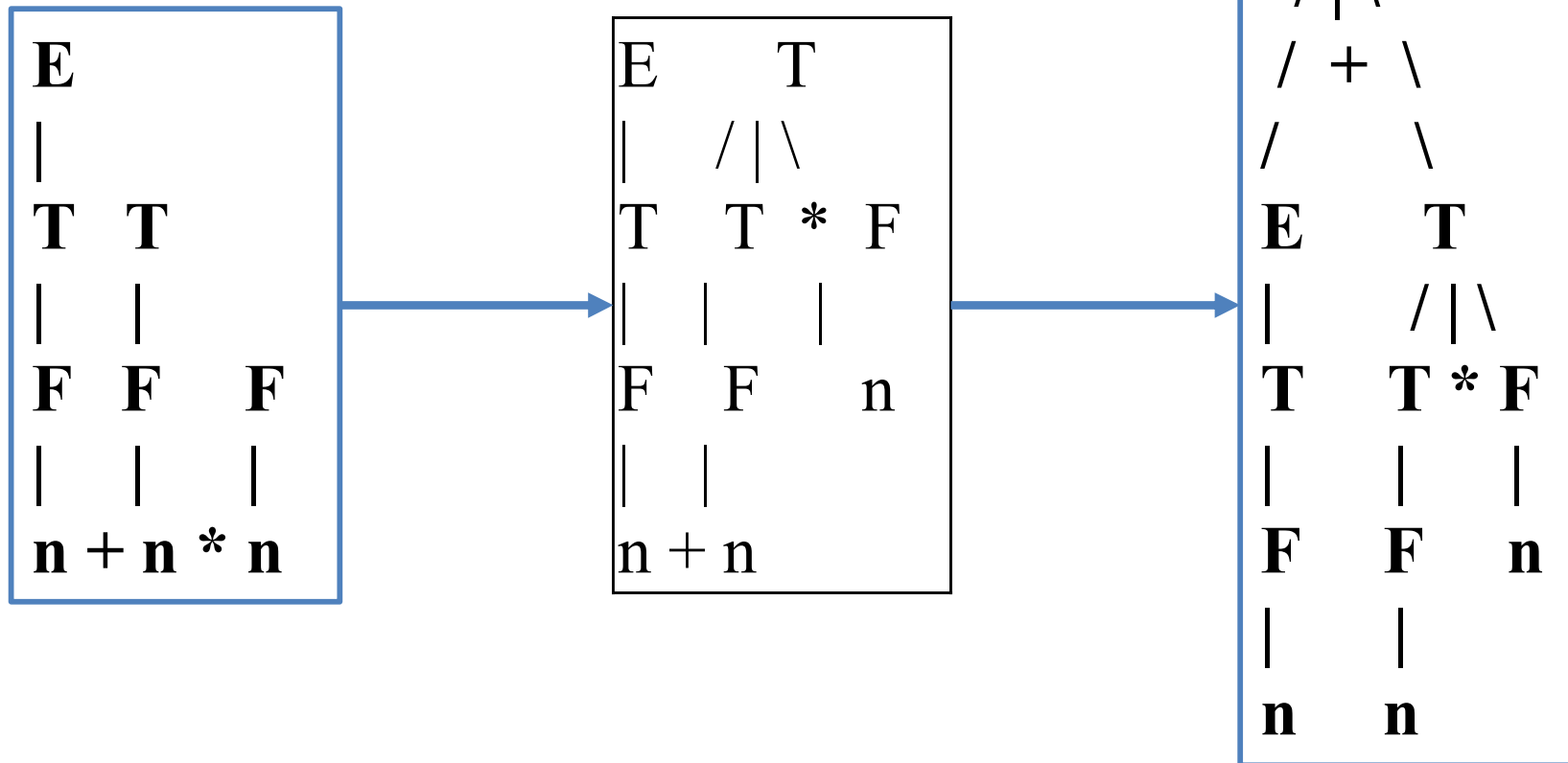
Bottom-up parse of $\mathbf{n + n * n}$ using the following grammar

$$\begin{array}{lll} E & \rightarrow & T \\ E & \rightarrow & E + T \\ T & \rightarrow & F \\ T & \rightarrow & T * F \\ F & \rightarrow & \mathbf{n} \\ F & \rightarrow & (E) \end{array}$$

Bottom-Up Parse Example



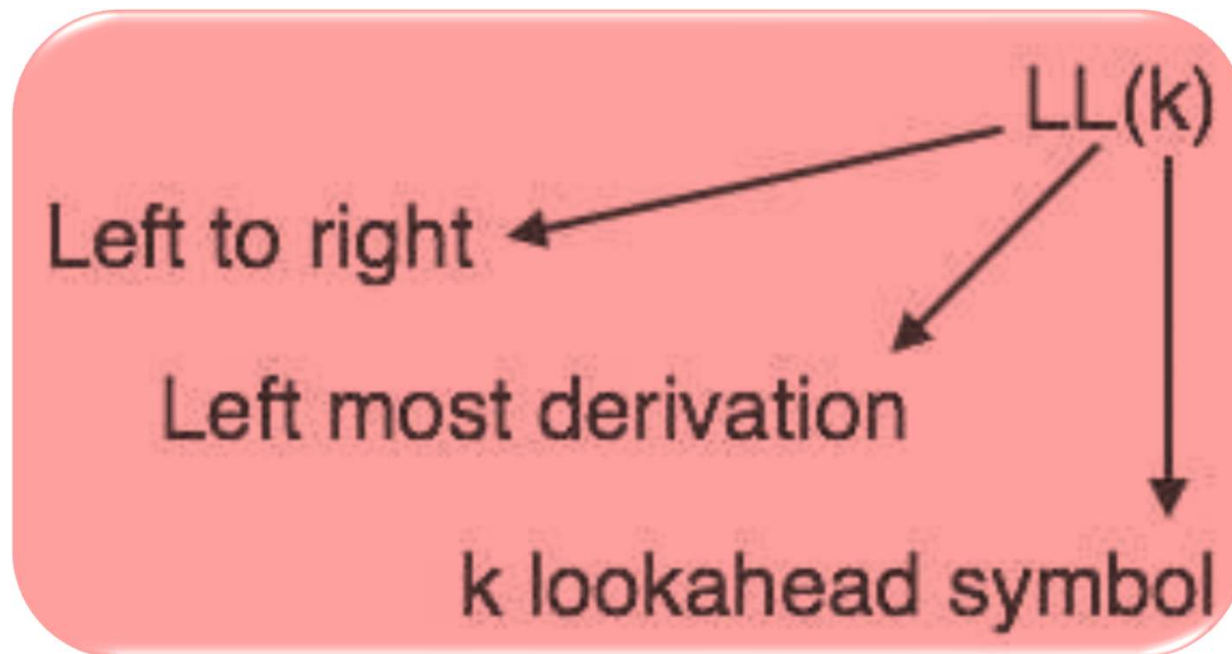
Bottom-Up Parse Example



LL (1) Parsing(CO2)

An LL Parser accepts LL grammar.

LL grammar can be implemented by means of both algorithms namely, recursive-descent or table-driven

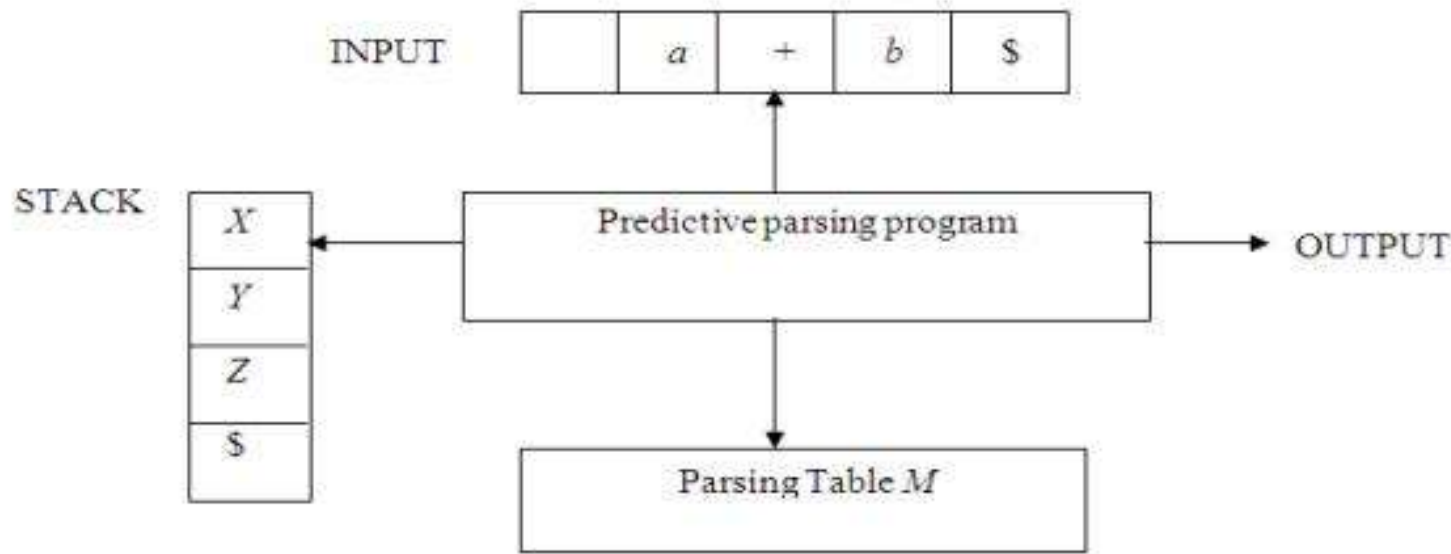


It is a type of parser which has the capability to predict which production is to be used to replace the input string.

To fulfil this tasks, the predictive parser uses a lookahead pointer, which points to the next symbols.

To make the parser back-tracking free, the predictive parser puts some constraints on the grammar and accepts only a class of grammar known as LL(K) grammar.

LL(1) Parser / Predictive parsing



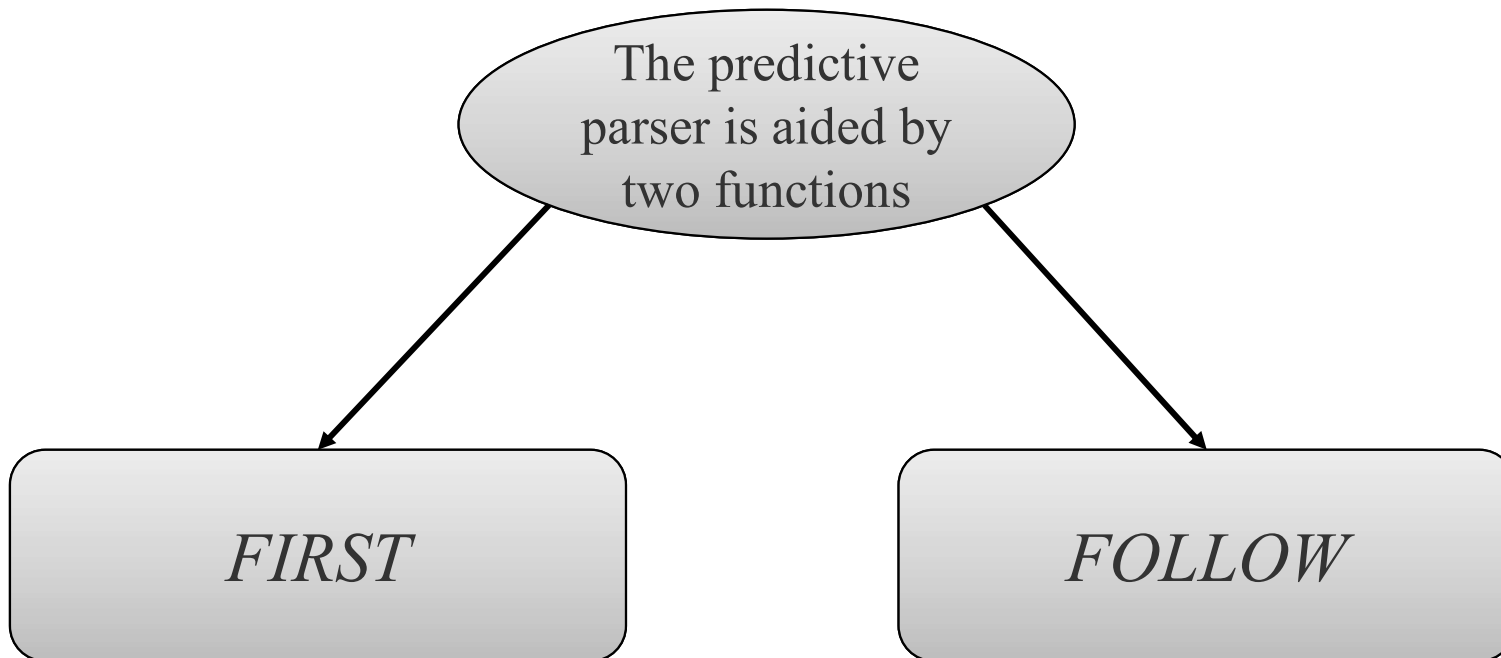
- It uses a stack and a parsing table to parse the input and generates a parse tree. Both the stack and the input contains an end symbol \$ to denote that the stack is empty and the input is consumed.
- The parser refers to the parsing table to take any decision on the input and stack element combination.

- *The input buffer contains the string to be parsed, followed by \$, a symbol used as a right end marker to indicate the end of the input string.*
- *The stack contains a sequence of grammar symbols with \$ on the bottom, indicating the bottom of the stack. Initially, the stack contains the start symbol of the grammar on top of \$.*
- *The parsing table is a two dimensional array $M[A,a]$ where A is a nonterminal, and a is a terminal or the symbol \$.*

A(X)\a	+	*	id	num	\$
E					
T					
T'					
F					
F'					

General Structure of parsing Table

Predictive parsing table construction



First (X):-

- It is used for finding the set of all terminals that begins the strings derive from the given grammar symbol.

Follow (X):-

- Set of Terminals that appear immediately to the right of X

To complete $\text{first}(x)$ for all grammar symbols X apply the following rules until no more terminal or ϵ be added to any first set.

Rules for $\text{first}(X)$:

1. If X is terminal, then

$$\text{FIRST}(X) = \{X\}.$$

2. If $X \rightarrow \epsilon$ is a production, then

$$\text{FIRST}(X) = \{\epsilon\}.$$

3. If X is non-terminal and $X \rightarrow a\alpha$ is a production then

$$\text{FIRST}(X) = \{a\}.$$

4. If X is non-terminal and $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production, then

$$\text{FIRST}(X) = \{\text{FIRST}(Y_1)\}.$$

If ϵ is in $\text{FIRST}(Y_1)$ then

$$\text{FIRST}(X) = \{\text{FIRST}(Y_2)\}.$$

Example on First Function

Example:- Calculate the first function of production rules:

$S \rightarrow ABCD$

$A \rightarrow a \mid \epsilon$

$B \rightarrow CD \mid b$

$C \rightarrow c \mid \epsilon$

$D \rightarrow Aa \mid d \mid \epsilon$

Solution:-

Symbols	First()
S	{a,b,c,d, ϵ }
A	{a, ϵ }
B	{c,a,d,b, ϵ }
C	{c, ϵ }
D	{a,d, ϵ }

Example on First Function

Example:- Calculate the first function of production rules:

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC / \epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g / \epsilon$$

$$F \rightarrow f / \epsilon$$

Solution:-

Symbols	First()
S	{a}
B	{c}
C	{b, ϵ }
D	{g, f ϵ }
E	{g, ϵ }
F	{f, ϵ }

Follow Function

To compute follows of A for all non-terminal A apply the following rules:-

1. If S is a start symbol,

$$\text{FOLLOW}(S) = \{\$ \}$$

2. If there is a production $A \rightarrow \alpha B \beta$,

$$\text{FOLLOW}(B) = \text{FIRST}(\beta) \text{ except } \epsilon$$

3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ , then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

$$\text{FOLLOW}(B) = \text{FOLLOW}(A)$$

Example:- Calculate the follow function of production rules:

$S \rightarrow ABCD$

$A \rightarrow a \mid \epsilon$

$B \rightarrow CD \mid b$

$C \rightarrow c \mid \epsilon$

$D \rightarrow Aa \mid d \mid \epsilon$

Solution:-

Symbols	First()
S	{a,b,c,d, ϵ }
A	{a, ϵ }
B	{c,a,d,b, ϵ }
C	{c, ϵ }
D	{a,d, ϵ }

Symbols	Follow()
S	{ ϵ }
A	{a,b,c,d, ϵ }
B	{c,a,d, ϵ }
C	{a,d,c, ϵ }
D	{c,a,d, ϵ }

Example on Follow Function

Example:- Calculate the follow function of production rules:

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC / \epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g / \epsilon$$

$$F \rightarrow f / \epsilon$$

Solution:-

Symbols	First()
S	{a}
B	{c}
C	{b, ϵ }
D	{g, f, ϵ }
E	{g, ϵ }
F	{f, ϵ }

Symbols	Follow()
S	{ $\$$ }
B	{g, f, h}
C	{g, f, h}
D	{h}
E	{f, h}
F	{h}

Left Recursion:-

A production of grammar is said to have **left recursion** if the leftmost variable of its RHS is same as variable of its LHS.

Example: $A \rightarrow A\alpha | \beta$

Right Recursion-

A production of grammar is said to have **right recursion** if the rightmost variable of its RHS is same as variable of its LHS

Example: $A \rightarrow \alpha A | \beta$

Left Recursion

Left recursion is eliminated by converting the grammar into a right recursive grammar.

If we have the left-recursive pair of productions:-

$$A \rightarrow A\alpha / \beta$$

where β does not begin with an A.

Then, we can eliminate left recursion by replacing the pair of productions with-

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' / \epsilon \end{aligned}$$

This right recursive grammar functions same as left recursive grammar.

Example: Left Recursion

Example1:- Consider the following grammar and eliminate left recursion-

$$A \rightarrow ABd / Aa / a$$

$$B \rightarrow Be / b$$

Solution:-

Firstly break the production according to the format-

$$A \rightarrow ABd / a$$

$$A \rightarrow Aa / a$$

$$B \rightarrow Be / b$$

Than apply the rule

$$A \rightarrow A\alpha / \beta \Rightarrow A \rightarrow \beta A' \\ A' \rightarrow \alpha A' / \epsilon$$

Example: Left Recursion

The grammar after eliminating left recursion is-

$A \rightarrow ABd / a$
 $A \rightarrow Aa / a$
 $B \rightarrow Be / b$



$A \rightarrow aA'$
 $A' \rightarrow BdA' / aA' / \epsilon$
 $B \rightarrow bB'$
 $B' \rightarrow eB' / \epsilon$

Example 2:- Consider the following grammar and eliminate left recursion-

$$E \rightarrow E + T / T$$

$$T \rightarrow T \times F / F$$

$$F \rightarrow \text{id}$$

Solution:- The grammar after eliminating left recursion is

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow \times FT' / \epsilon$$

$$F \rightarrow \text{id}$$

Example of first and follow function with left recursion

Example:- Calculate the first and follow function of production rules:

$$S \rightarrow A$$

$$A \rightarrow aB / Ad$$

$$B \rightarrow b$$

$$C \rightarrow g$$

Solution:-

Given grammar is left recursive.

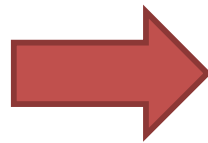
- So, first remove left recursion from the given grammar.

Example of first and follow function with left recursion

After eliminating left recursion, we get the following grammar

- $S \rightarrow A$
- $A \rightarrow aBA'$
- $A' \rightarrow dA' / \epsilon$
- $B \rightarrow b$
- $C \rightarrow g$

First Function



Symbols	First()
S	{a}
A	{a}
A'	{d, ϵ }
B	{b}
C	{g}

Example of first and follow function with left recursion

Follow Function:-

Symbols	Follow()
S	{ $\$$ }
A	{ $\$$ }
A'	{ $\$$ }
B	{d, $\$$ }
C	{NA}

Left factoring is a process by which the grammar with common prefixes is transformed.

To make it useful for Top down parsers.

Example- $A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$

- Top down parsers can not decide which production must be chosen to parse the string in hand.
- To remove this confusion, we use left factoring.

Left Factoring

In left factoring,

- We make one production for each common prefixes.
- The common prefix may be a terminal or a non-terminal or a combination of both.
- Rest of the derivation is added by new productions.

$$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$$



$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 / \beta_2 / \beta_3 \end{aligned}$$

Example of Left Factoring

Example: Do left factoring in the following grammar-

$$S \rightarrow iEtS / iEtSeS / a$$

$$E \rightarrow b$$

Solution-

The left factored grammar is-

$$S \rightarrow iEtSS' / a$$

$$S' \rightarrow eS / \epsilon$$

$$E \rightarrow b$$

Example of Left Factoring

Example: Do left factoring in the following grammar-

$$A \rightarrow aAB / aBc / aAc$$

Solution-

Step-01:

$$A \rightarrow aA'$$

$$A' \rightarrow AB / Bc / Ac$$

Again, this is a grammar with common prefixes.

Step-02:

$$A \rightarrow aA'$$

$$A' \rightarrow AD / Bc$$

$$D \rightarrow B / c$$

This is a left factored grammar.

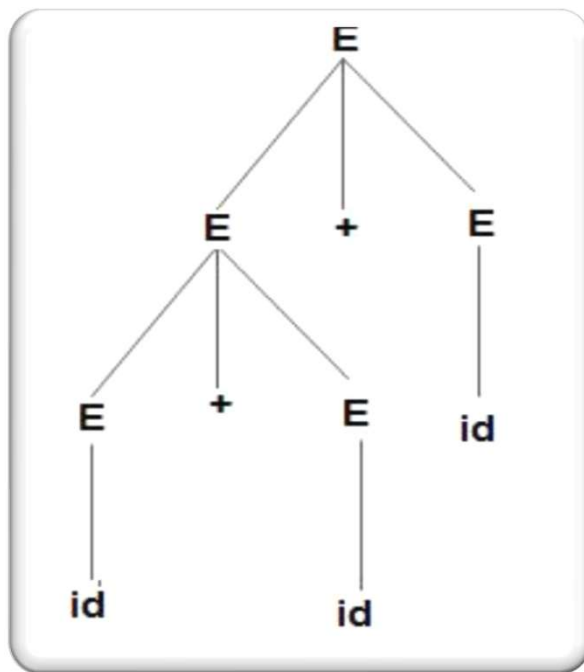
Example: Production rule-

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

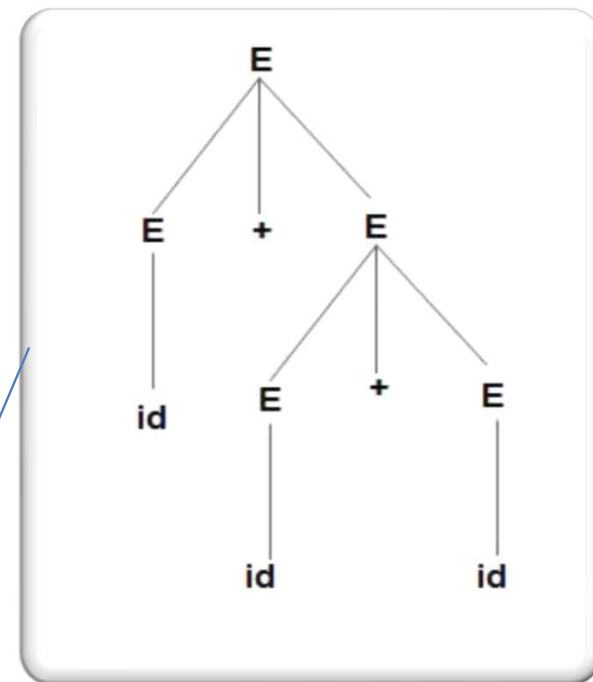
$$E \rightarrow id$$

Input string:-id+id+id



Correct

Incorrect
Bcoz the rule
of associativity
is failed



Example: Production rule-

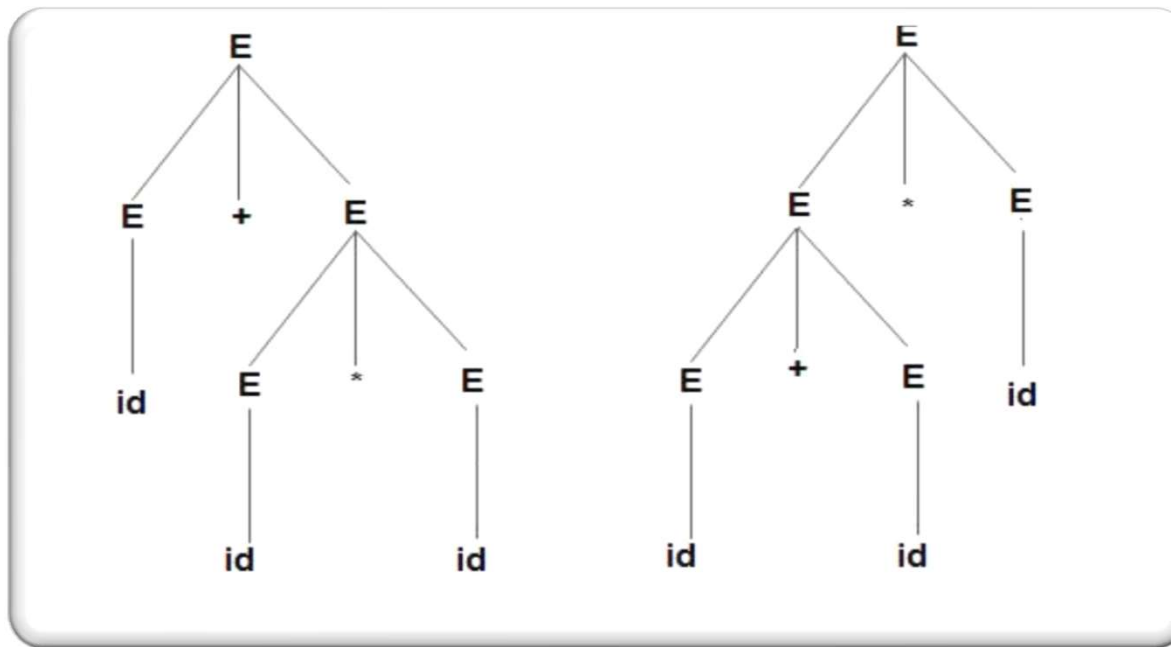
$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

Input string:-id+id*id

Correct



Incorrect
Because
precedence is
not taken care
of

Left Recursion, Common Prefixes makes the grammar Ambiguous

An ambiguous grammar may be converted into an unambiguous grammar by implementing

- Precedence Constraints:-
- Associativity Constraints

Associativity Rule:

- If the operator is left associative, induce left recursion in its production.
- If the operator is right associative, induce right recursion in its production.

Ambiguity

According to the associativity rule:- Left associativity is obtain by converting the grammar into left recursion.

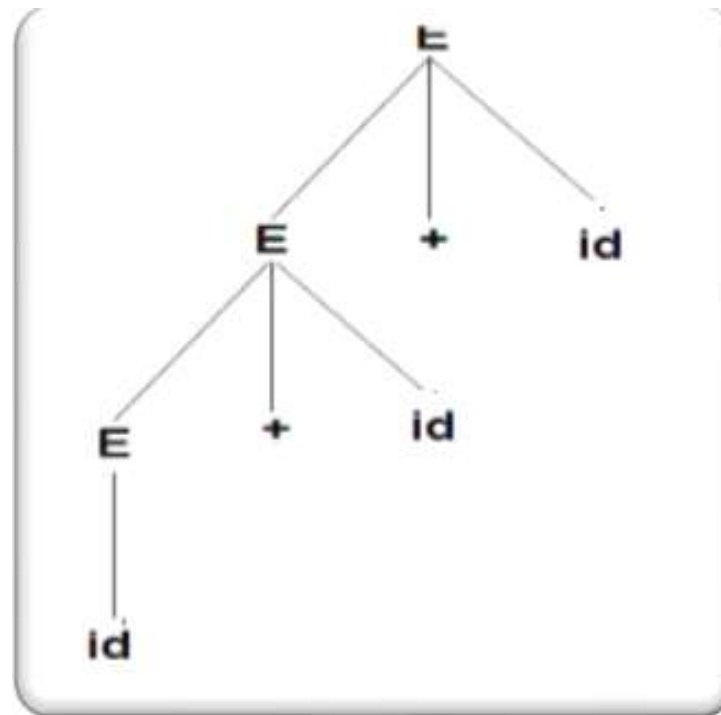
Production rule: $E \rightarrow E + E$

$E \rightarrow E + E$

$E \rightarrow id$

$E \rightarrow E + id$

$E \rightarrow id$



Precedence Rule:

- The level at which the production is present defines the priority of the operator contained in it.
- The higher the level of the production, the lower the priority of operator.
- The lower the level of the production, the higher the priority of operator.

Ambiguity

According to the Precedence rule:- Highest precedence operator should be at the least level.

Production rule: $E \rightarrow E + E$

$E \rightarrow E * E$

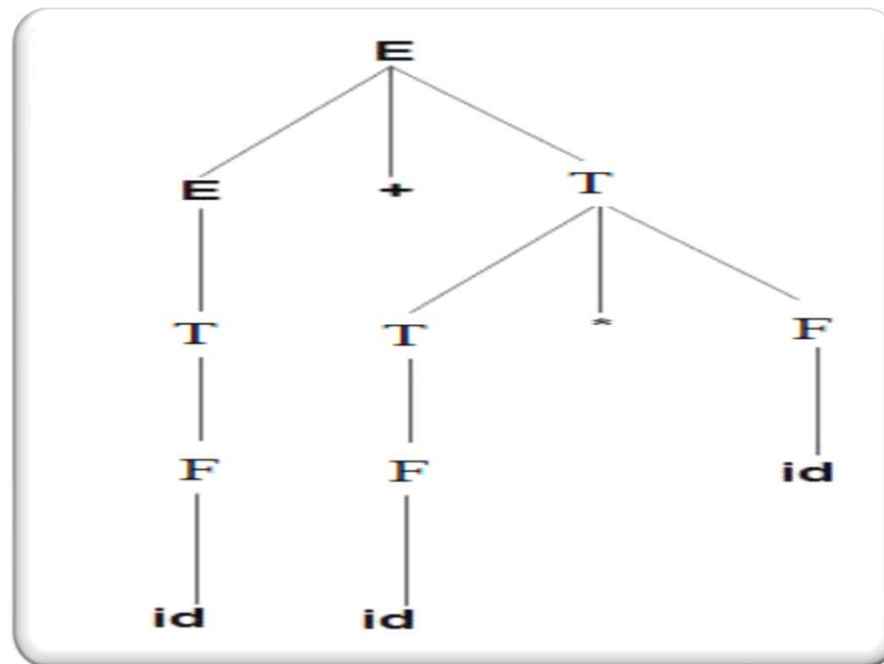
$E \rightarrow id$



$E \rightarrow E + T | T$

$T \rightarrow T * F | F$

$F \rightarrow id$



Example: Consider the following grammar :

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E) | id$$

Check whether this input string $id + id * id$ is accepted using LL(1) parsing technique

Solution:

Step 1: Check whether a given grammar is ambiguous or not.

If ambiguous remove ambiguity:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Step 2:

Check and Remove Left –Recursion

After eliminating left-recursion the grammar is

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

$$\begin{aligned} A \rightarrow A\alpha / \beta &\Rightarrow A \rightarrow \beta A' \\ A' &\rightarrow \alpha A' / \epsilon \end{aligned}$$

Step 3:

Check and Remove Left –Factoring

$$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$$



$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 / \beta_2 / \beta_3 \end{aligned}$$

There is no left factoring.

Step 4: Construct predictive parse table.

Calculate first and follow function

First function

$$\text{FIRST}(E) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

Follow function

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ +, \$,) \}$$

$$\text{FOLLOW}(T') = \{ +, \$,) \}$$

$$\text{FOLLOW}(F) = \{ +, *, \$,) \}$$

Predictive Parse Table

NON- TERMINAL	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Example

Stack Implementation:

stack	Input	Output
SE	<u>id+id</u> *id S	
SE'T	<u>id+id</u> *id S	$E \rightarrow TE'$
SE'T'F	<u>id+id</u> *id S	$T \rightarrow FT'$
SE'T' <u>id</u>	<u>id+id</u> *id S	<u>$F \rightarrow id$</u>
SE'T'	+id*id S	
SE'	+id*id S	$T' \rightarrow \epsilon$
SE'T+	+id*id S	$E' \rightarrow +TE'$
SE'T	id*id S	
SE'T'F	id*id S	$T \rightarrow FT'$
SE'T' <u>id</u>	id*id S	<u>$F \rightarrow id$</u>
SE'T'	*id S	
SE'T'F*	*id S	$T' \rightarrow *FT'$
SE'T'F	id S	
SE'T' <u>id</u>	id S	<u>$F \rightarrow id$</u>
SE'T'	S	
SE'	S	$T' \rightarrow \epsilon$
S	S	$E' \rightarrow \epsilon$

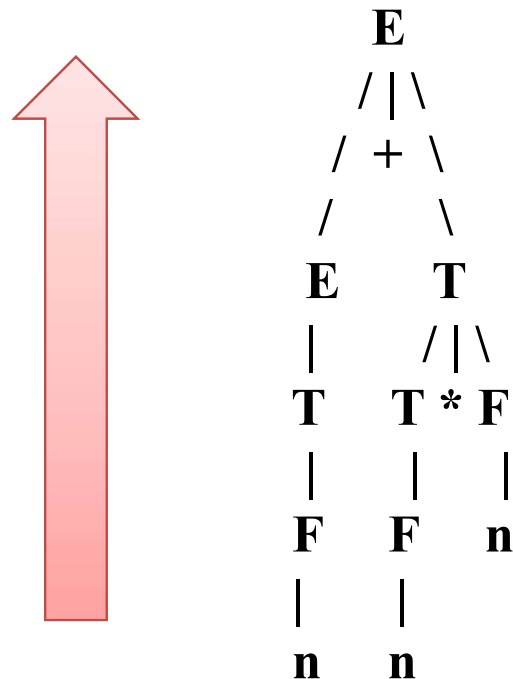
Implementation of predictive parser:

1. Elimination of left recursion, left factoring and ambiguous grammar.
2. Construct FIRST() and FOLLOW() for all non-terminals.
3. Construct predictive parsing table.
4. Parse the given input string using stack and parsing table

Bottom-up Parsing(CO2)

Bottom-up parsing starts from the leaf nodes of a tree and works in upward direction till it reaches the root node.

Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol



Shift-Reduce Parsing(CO2)

Shift-Reduce Parsing uses two unique steps for bottom-up parsing. These steps are known as shift-step and reduce-step.

Shift step:

- The shift step refers to the advancement of the input pointer to the next input symbol, which is called the shifted symbol.
- This symbol is pushed onto the stack.
- The shifted symbol is treated as a single node of the parse tree.

Reduce step :

- When the parser finds a complete grammar rule (RHS) and replaces it to (LHS), it is known as reduce-step.
- This occurs when the top of the stack contains a handle.
- To reduce, a POP function is performed on the stack which pops off the handle and replaces it with LHS non-terminal symbol.

Shift-Reduce Parsing

Shift Action, the current symbol in the input string pushed into a stack.

At each reduction, the symbol will be replaced by the non-terminals.

A String $\xrightarrow{\text{reduce to}}$ the starting symbol

Example of Shift-Reduce Parsing

Example:

Grammar: $S \rightarrow S+S$
 $S \rightarrow S-S$
 $S \rightarrow (S)$
 $S \rightarrow a$

Input String: $a_1-(a_2+a_3)$

Perform Shift –Reduce Parsing on above grammar.

Example of Shift-Reduce Parsing

Stack contents	Input string	Actions
\$	a1-(a2+a3)\$	shift a1
\$a1	-(a2+a3)\$	reduce by $S \rightarrow a$
\$S	-(a2+a3)\$	shift -
\$S-	(a2+a3)\$	shift (
\$S-(	a2+a3)\$	shift a2
\$S-(a2	+a3)\$	reduce by $S \rightarrow a$
\$S-(S	+a3) \$	shift +
\$S-(S+	a3) \$	shift a3
\$S-(S+a3	) \$	reduce by $S \rightarrow a$
\$S-(S+S	) \$	shift)
\$S-(S+S)	\$	reduce by $S \rightarrow S+S$
\$S-(S)	\$	reduce by $S \rightarrow (S)$
\$S-S	\$	reduce by $S \rightarrow S-S$
\$S	\$	Accept

Example of Shift-Reduce Parsing

Example:

Grammar: $E \rightarrow 2E2$
 $E \rightarrow 3E3$
 $E \rightarrow 4$

Input String: 32423

Perform Shift –Reduce Parsing on above grammar.

Example of Shift-Reduce Parsing

Stack	Input Buffer	Parsing Action
\$	32423\$	Shift
\$3	2423\$	Shift
\$32	423\$	Shift
\$324	23\$	Reduce by E --> 4
\$32E	23\$	Shift
\$32E2	3\$	Reduce by E --> 2E2
\$3E	3\$	Shift
\$3E3	\$	Reduce by E --> 3E3
\$E	\$	Accept

The LR(k) parsing technique

- ★ L is for Left-to-right scanning of input
- ★ R corresponds to a Rightmost derivation done in reverse
- ★ k is the number of lookahead symbols used to make parsing decisions

Types of LR Parser(CO2)

Four LR parsing techniques will be considered

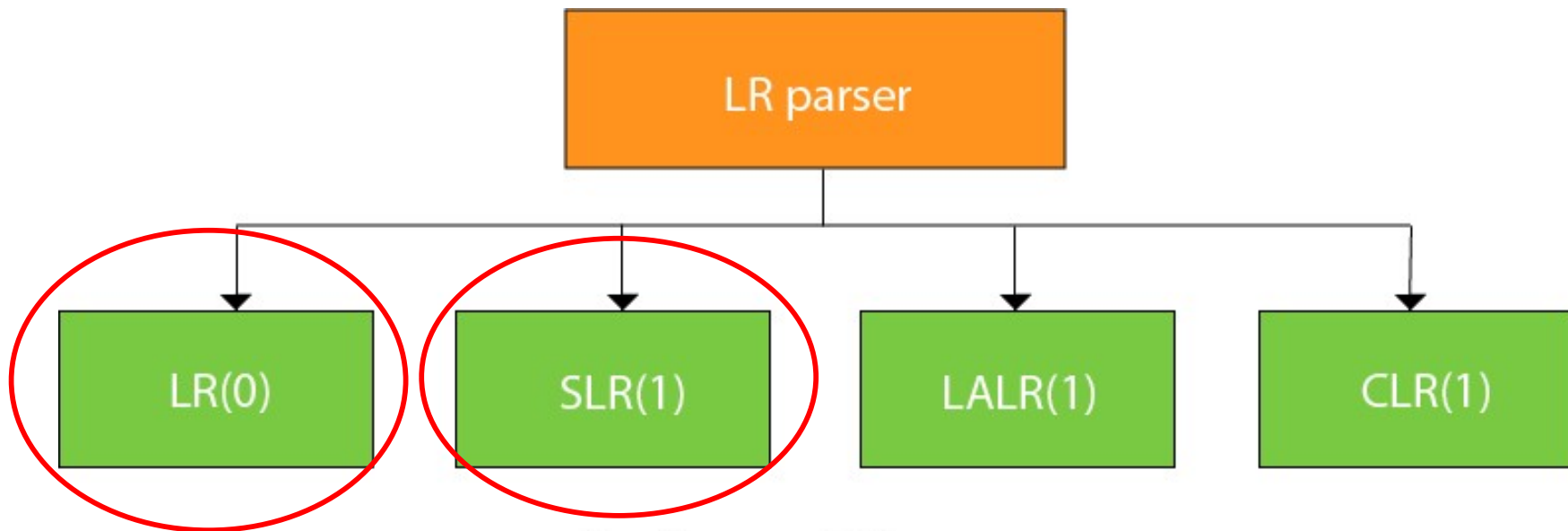


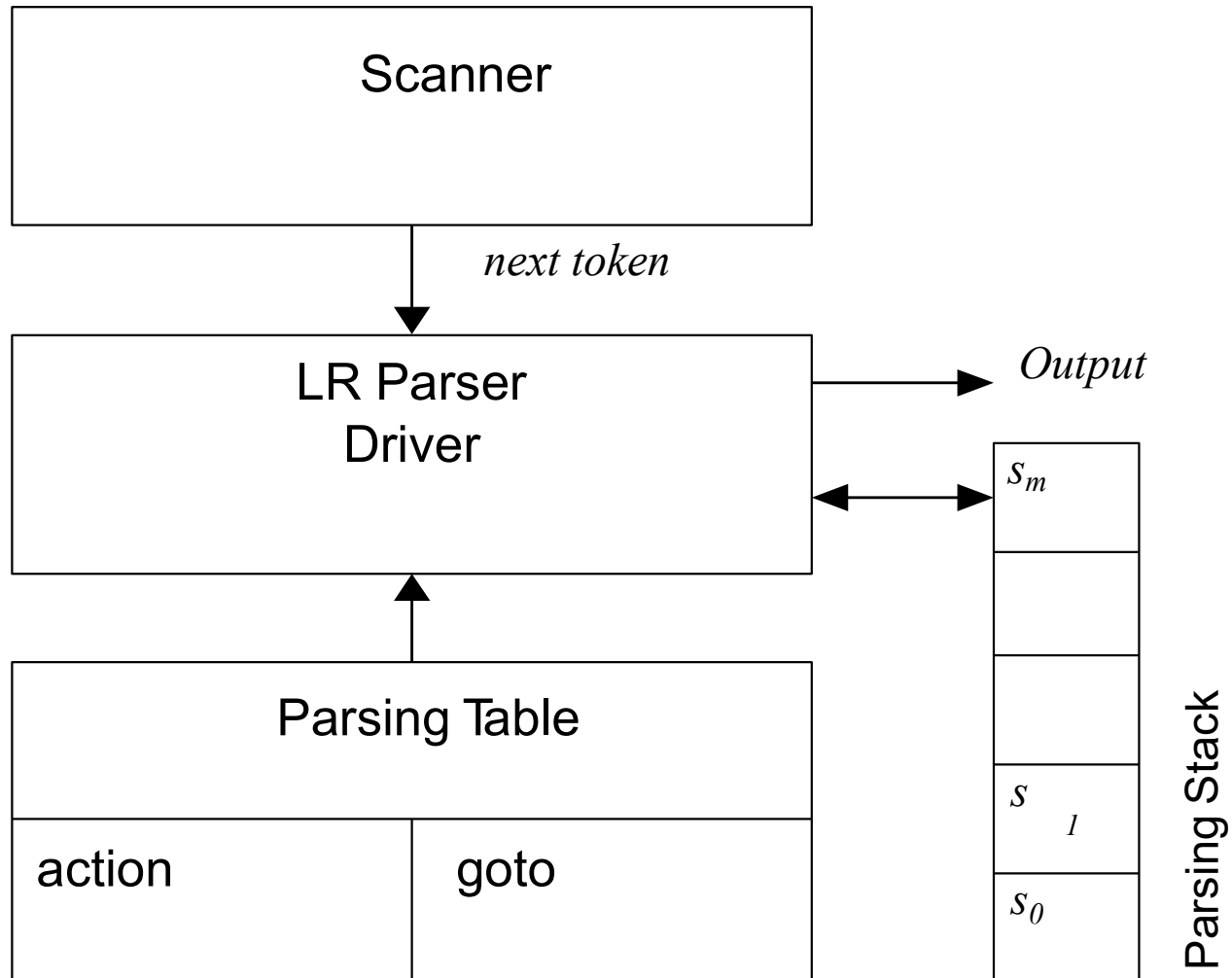
Fig: Types of LR parser

Types of LR Parser

- LR(0)-LR parsing with no lookahead token to make parsing decisions
- SLR(1)-Simple LR, with one token of lookahead
- CLR(1)- Canonical LR, with one token of lookahead
- LALR(1) : Lookahead LR, with one token of lookahead

LALR(1) is the preferable technique used by parser generators

Structure of LR Parser



An LR parser consists of ...

Driver program

- Same driver is used for all LR parsers

Parsing stack

- Contains state information, where s_i is *state i*
- States are obtained from grammar analysis

Parsing table, which has two parts

- Action section: specifies the parser actions
- Goto section: specifies the successor states

Various steps involved in the LR (0) Parsing

For the given input string write a context free grammar

Check the ambiguity of the grammar

Add Augment production in the given grammar

Create Canonical collection of LR (0) items

Draw a data flow diagram (DFD)

Construct a LR (0) parsing table

Example:

Question:

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Solution:

The Augment grammar G' is represented by

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Canonical Collection of LR(0) items

- An LR (0) item is a production G with dot at some position on the right side of the production.
- LR(0) items is useful to indicate that how much of the input has been scanned up to a given point in the process of parsing.
- In the LR (0), we place the reduce node in the entire row.

Example:-

$$S \rightarrow AA \quad \& \quad A \rightarrow aA \mid b$$

Solution:

Add Augment Production and insert ' $\bullet$ ' symbol at the first position for $S' \rightarrow \bullet S$ production.

$$S' \rightarrow \bullet S$$

$$S \rightarrow \bullet AA$$

$$A \rightarrow \bullet aA$$

$$A \rightarrow \bullet b$$

Create Canonical Collection of LR(0) items

I0 State:

Add Augment production to the I0 State and Compute the Closure

$I0 = \text{Closure}(S' \rightarrow \bullet S)$

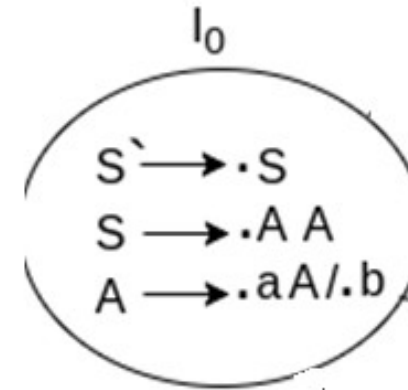
Add all productions starting with S in to I0 State because " $\bullet$ " is followed by the non-terminal. So, the I0 State becomes

$I0 = S' \rightarrow \bullet S$
 $S \rightarrow \bullet AA$

Add all productions starting with "A" in modified I0 State because " $\bullet$ " is followed by the non-terminal. So, the I0 State becomes.

Create Canonical Collection of LR(0) items

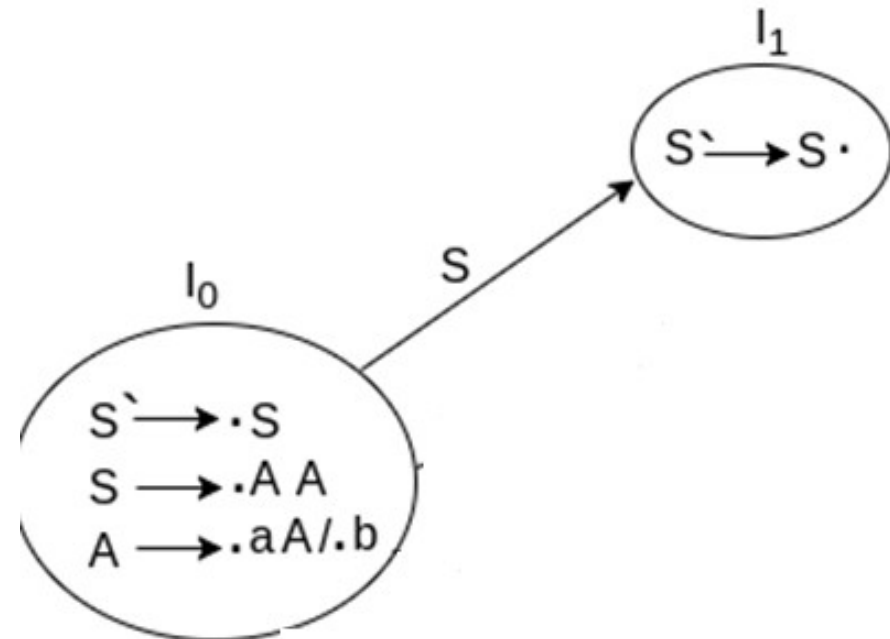
$I_0 = S' \rightarrow \cdot S$
 $S \rightarrow \cdot AA$
 $A \rightarrow \cdot aA$
 $A \rightarrow \cdot b$



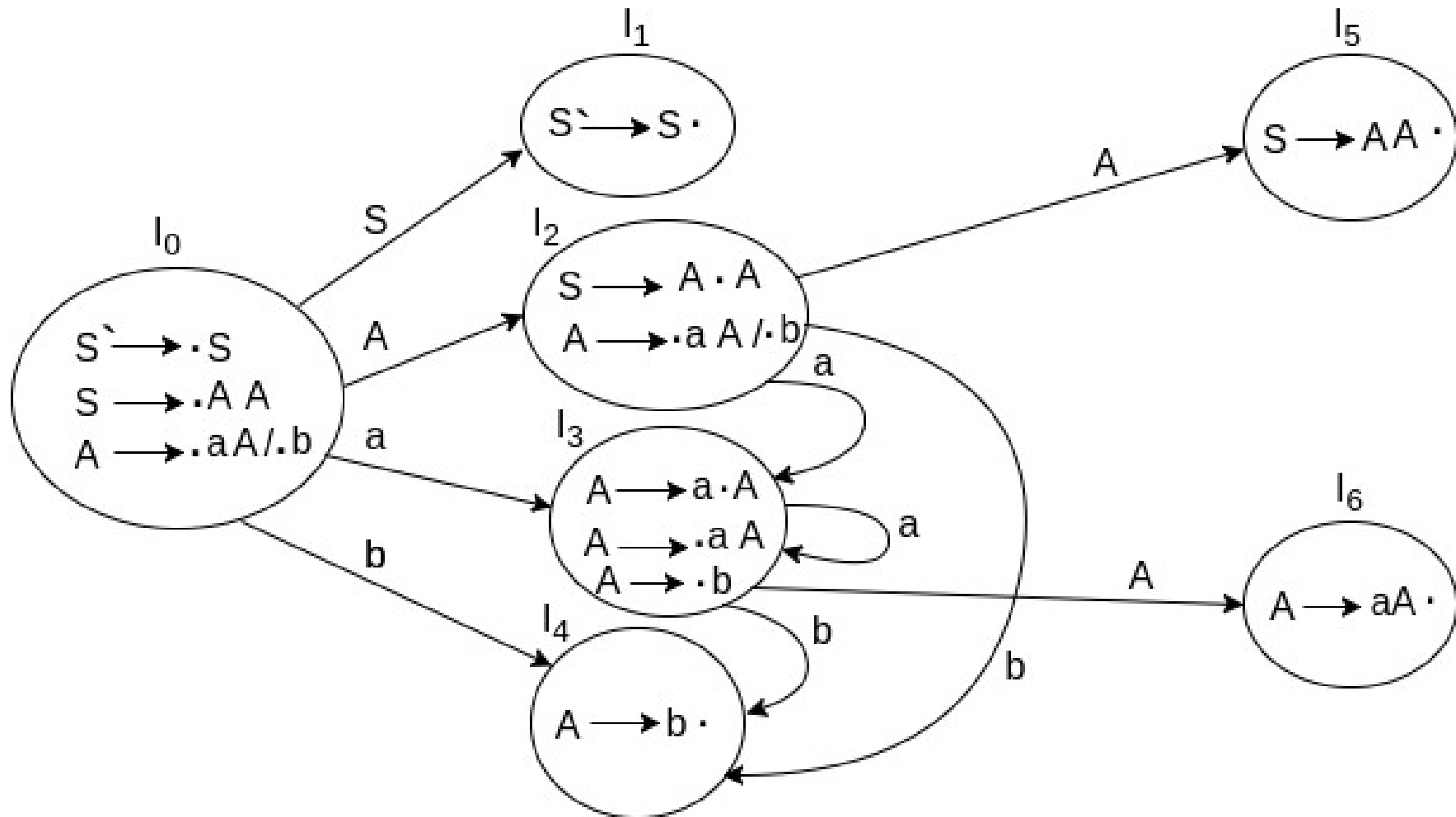
$I_1 = \text{Go to } (I_0, S) = \text{closure } (S' \rightarrow S \cdot) = S' \rightarrow S \cdot$

Here, the Production is reduced so close the State.

$I_1 = S' \rightarrow S \cdot$



DRAWING DFD



For I2, consider 2nd production in I0 $S \rightarrow \bullet AA$

Go to (I0, A) = closure ($S \rightarrow A\bullet A$)

Add all productions starting with A in to I2 State because " $\bullet$ " is followed by the non-terminal. So, the I2 State becomes

I2 = $S \rightarrow A\bullet A$

$A \rightarrow \bullet aA$

$A \rightarrow \bullet b$

- Go to (I2, a) = Closure ($A \rightarrow a\bullet A$) = (same as I3)
- Go to (I2, b) = Closure ($A \rightarrow b\bullet$) = (same as I4)

Canonical Collection of LR(0) items

I3 = Go to (I0, a) = Closure ($A \rightarrow a \bullet A$)

Add productions starting with A in I3.

- $A \rightarrow a \bullet A$
 $A \rightarrow \bullet a A$
 $A \rightarrow \bullet b$
- Go to (I3, a) = Closure ($A \rightarrow a \bullet A$) = (same as I3)
Go to (I3, b) = Closure ($A \rightarrow b \bullet$) = (same as I4)

Canonical Collection of LR(0) items

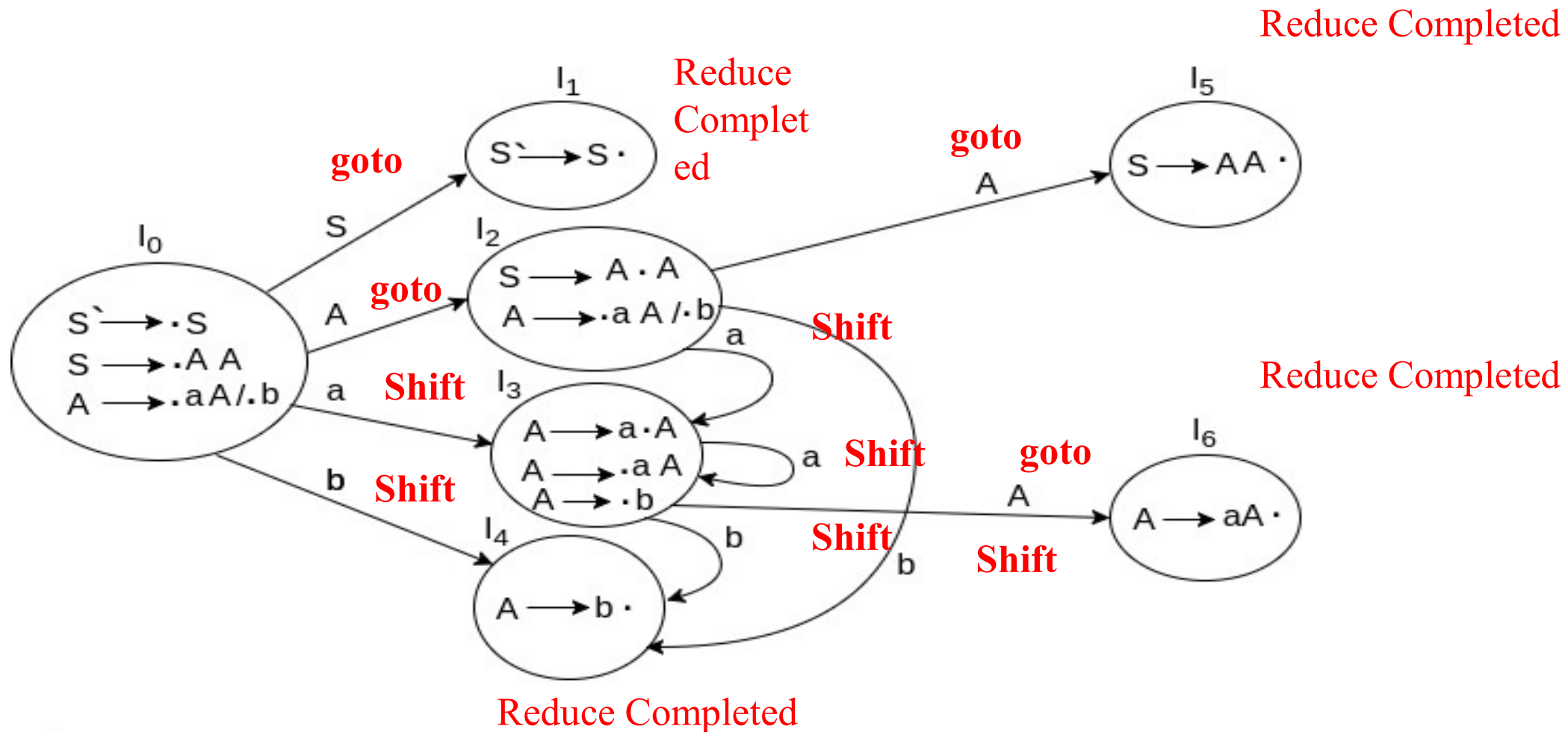
I4= Go to (I0, b) = closure ($A \rightarrow b\bullet$) = $A \rightarrow b\bullet$

I5= Go to (I2, A) = Closure ($S \rightarrow AA\bullet$) = $SA \rightarrow A\bullet$

I6= Go to (I3, A) = Closure ($A \rightarrow aA\bullet$) = $A \rightarrow aA\bullet$

- If a state is going to some other state on a terminal then it correspond to a shift move.
- If a state is going to some other state on a variable then it correspond to go to move.
- If a state contain the final item in the particular row then write the reduce node completely

Drawing DFD



LR(0) Table

States	Action			Go to	
	a	b	\$	A	S
I ₀	S3	S4		2	1
I ₁	accept				
I ₂	S3	S4		5	
I ₃	S3	S4		6	
I ₄	r3	r3	r3		
I ₅	r1	r1	r1		
I ₆	r2	r2	r2		

Productions are numbered as follows:

- $S \rightarrow AA \dots (1)$
- $A \rightarrow aA \dots (2)$
- $A \rightarrow b \dots (3)$

LR(0) Table- Explanation

- I1 contains the final item which drives($S' \rightarrow S\bullet$), so action $\{I1, \$\} = \text{Accept}$.
- I4 contains the final item which drives $A \rightarrow b\bullet$ and that production corresponds to the production number 3 so write it as r3 in the entire row.
- I5 contains the final item which drives $S \rightarrow AA\bullet$ and that production corresponds to the production number 1 so write it as r1 in the entire row.
- I6 contains the final item which drives $A \rightarrow aA\bullet$ and that production corresponds to the production number 2 so write it as r2 in the entire row.

LR(0) Table-Explanation:

- I0 on S is going to I1 so write it as 1.
- I0 on A is going to I2 so write it as 2.
- I2 on A is going to I5 so write it as 5.
- I3 on A is going to I6 so write it as 6.
- I0, I2 and I3 on a are going to I3 so write it as S3 which means that shift 3.
- I0, I2 and I3 on b are going to I4 so write it as S4 which means that shift 4.
- I4, I5 and I6 all states contains the final item because they contain • in the right most end. So rate the production as production number.

SLR (1) refers to simple LR Parsing.

It is same as LR(0) parsing.

The only difference is in the parsing table.

To construct SLR (1) parsing table, we use canonical collection of LR (0) item.

In the SLR (1) parsing, we place the reduce move only in the follow of left hand side

Various steps involved in the SLR (1) Parsing

For the given input string write a context free grammar

Check the ambiguity of the grammar

Add Augment production in the given grammar

Create Canonical collection of LR (0) items

Draw a data flow diagram (DFD)

Construct a SLR (1) parsing table

SLR (1) Table Construction

The steps which use to construct SLR (1) Table is given below:

- If a state (I_i) is going to some other state (I_j) on a terminal then it corresponds to a shift move in the action part
- If a state (I_i) is going to some other state (I_j) on a variable then it correspond to go to move in the Go to part.
- If a state (I_i) contains the final item, which has no transitions to the next state then the production is known as reduce production. For all terminals X in FOLLOW (A), write the reduce entry along with their production numbers

Example 1:-

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow \text{id} \end{aligned}$$

Solution:-

Step 1:- Add Augment Production and insert ' $\bullet$ ' symbol at the first position of Augment Production

$$S \rightarrow E$$
$$S' \rightarrow \bullet E$$

Step 2: Compute Canonical Collection

I0 State:

$S' \rightarrow \bullet E$

I0 = Closure ($S' \rightarrow \bullet E$)

$S' \rightarrow \bullet E$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$



$S' \rightarrow \bullet E$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet id$

Example-SLR(1)

I1= Go to (I0, E)

$S' \rightarrow E \bullet$
 $E \rightarrow E \bullet + T$

I2= Go to (I0, T)

$E \rightarrow T \bullet$
 $T \rightarrow T \bullet * F$

I3= Go to (I0, F)

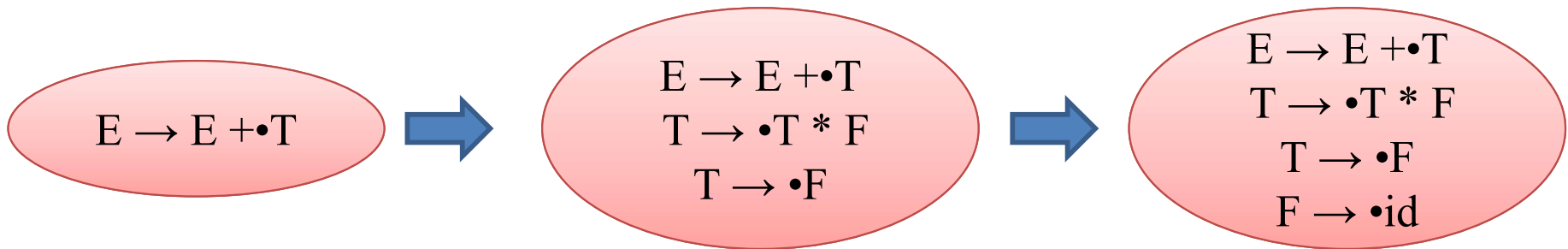
$T \rightarrow F \bullet$

I4= Go to (I0, id)

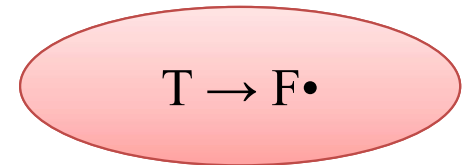
$F \rightarrow id \bullet$

Example-SLR(1)

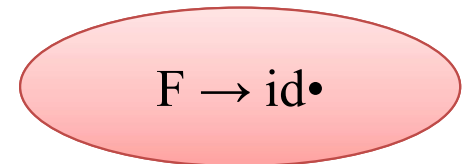
I5 = Go to (I1, +)



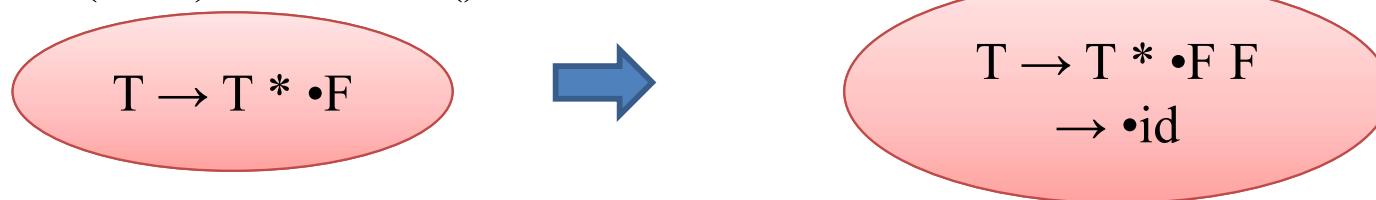
Go to (I5, F) = Closure ($T \rightarrow F \bullet$) = (same as I3)



Go to (I5, id) = Closure ($F \rightarrow id \bullet$) = (same as I4)



I6 = Go to (I2, *) = Closure ()



Example-SLR(1)

I6 = Go to (I6, id) = Closure ($F \rightarrow id\bullet$) = (same as I4)

$F \rightarrow id\bullet$

I7 = Go to (I5, T)

$E \rightarrow E + T\bullet$



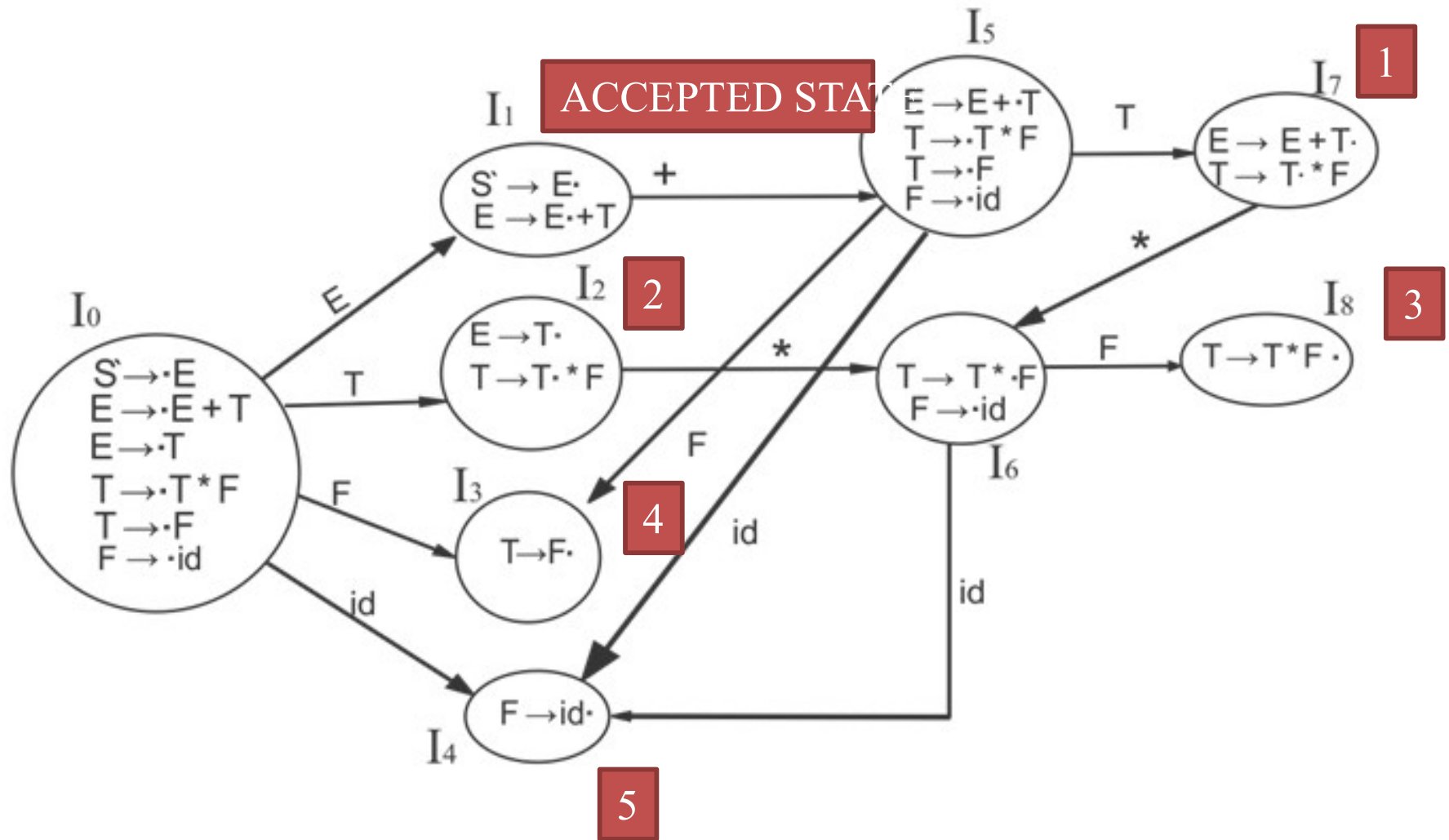
$E \rightarrow E + T\bullet$
 $T \rightarrow T\bullet * F$

I8 = Go to (I6, F)

$T \rightarrow T * F\bullet$

Example-SLR(1)

Step-3 Drawing DFD:



Productions are numbered as follows:

$E \rightarrow E + T$ (1)

$E \rightarrow T$ (2)

$T \rightarrow T * F$ (3)

$T \rightarrow F$ (4)

$F \rightarrow id$ (5)

First and Follow Function

$\text{First}(E) = \text{First}(E) \cup \text{First}(T)$

$\text{First}(T) = \text{First}(T) \cup \text{First}(F)$

$\text{First}(F) = \{\text{id}\}$

$\text{First}(T) = \{\text{id}\}$

$\text{First}(E) = \{\text{id}\}$

$\text{Follow}(E) = \text{First}(+T) \cup \{\$\} = \{+, \$\}$

$\text{Follow}(T) = \text{First}(*F) \cup \text{First}(F) \cup \text{Follow}(E) = \{*, +, \$\}$

$\text{Follow}(F) = \{*, +, \$\}$

Example-SLR(1)

Step-4 SLR (1) Table

States	Action				Go to		
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁	S ₅			Accept			
I ₂	R ₂	S ₆	R ₂				
I ₃	R ₄	R ₄	R ₄				
I ₄	R ₅	R ₅	R ₅				
I ₅	S ₄					7	3
I ₆	S ₄						8
I ₇	R ₁	S ₆	R ₁				
I ₈	R ₃	R ₃	R ₃				

Example-SLR(1)

- I1 contains the final item which drives $S \rightarrow E\bullet$ and follow (S) = { $\$$ }, so action {I1, $\$$ } = Accept
- I2 contains the final item which drives $E \rightarrow T\bullet$ and follow (E) = { $+$, $\$$ }, so action {I2, $+$ } = R2, action {I2, $\$$ } = R2
- I3 contains the final item which drives $T \rightarrow F\bullet$ and follow (T) = { $+$, $*$, $\$$ }, so action {I3, $+$ } = R4, action {I3, $*$ } = R4, action {I3, $\$$ } = R4
- I4 contains the final item which drives $F \rightarrow id\bullet$ and follow (F) = { $+$, $*$, $\$$ }, so action {I4, $+$ } = R5, action {I4, $*$ } = R5, action {I4, $\$$ } = R5
- I7 contains the final item which drives $E \rightarrow E + T\bullet$ and follow (E) = { $+$, $\$$ }, so action {I7, $+$ } = R1, action {I7, $\$$ } = R1
- I8 contains the final item which drives $T \rightarrow T * F\bullet$ and follow (T) = { $+$, $*$, $\$$ }, so action {I8, $+$ } = R3, action {I8, $*$ } = R3, action {I8, $\$$ } = R3.

Question:

$$A \rightarrow (A)/a$$

Solution:

Step 1:- Augmented Grammar

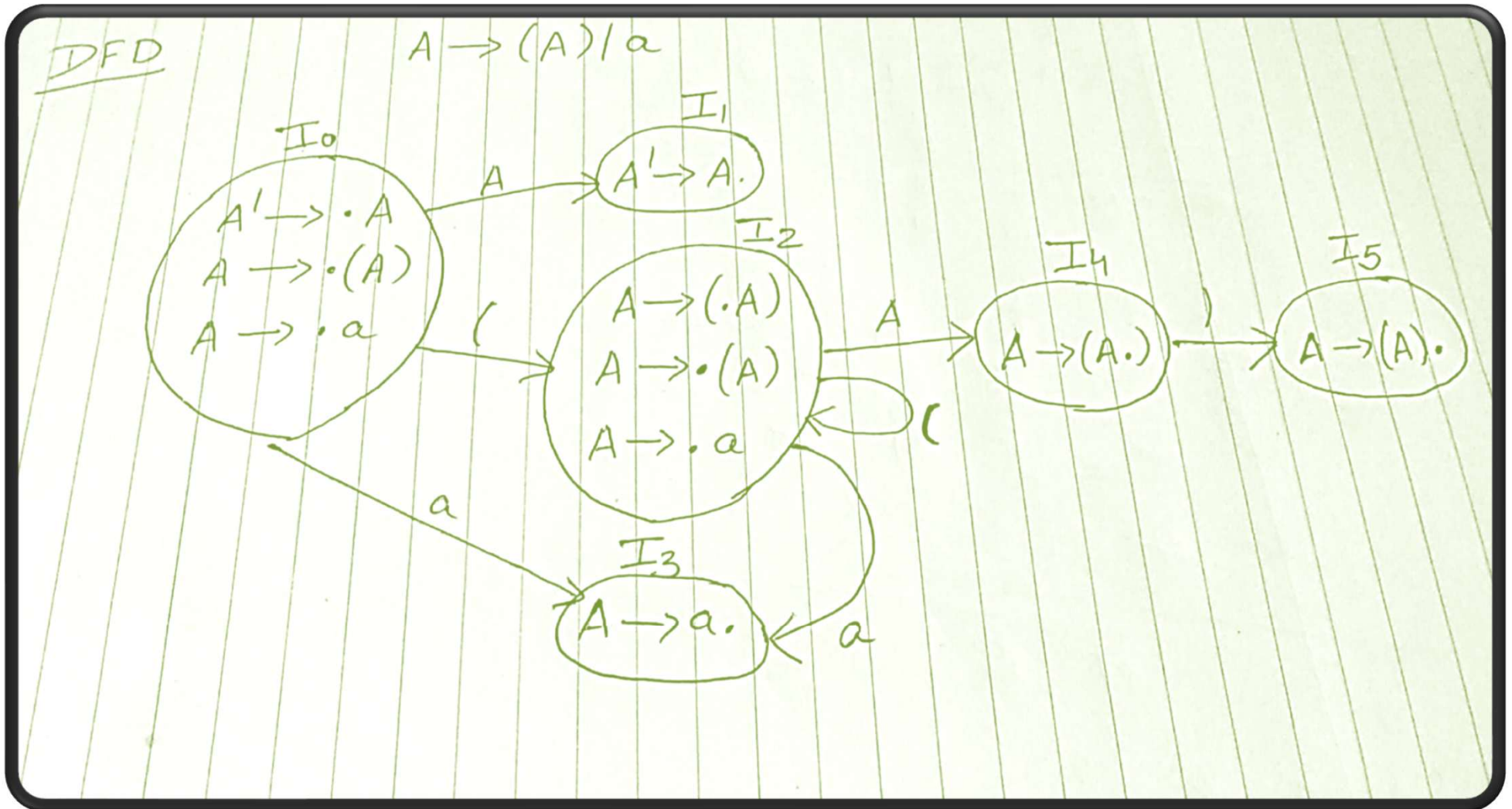
$$\begin{array}{l} A' \rightarrow A \\ A' \rightarrow .A \\ A \rightarrow .(A) \\ A \rightarrow .a \end{array}$$

Step 2:- Create Canonical Collection of LR(0) items

- Find out the canonical collection of each production.

Drawing DFD:

Step:-3 Drawing DFD:



Productions are numbered as follows:

$A \rightarrow (A).$ (1)

$A \rightarrow a.$ (2)

Calculate the follow function:

Follow of A will be {), \$ }

So, just place production number on the follow of A.

SLR (1) Table

State	Action	GoTo
	a () \$	A
0	S3 S2	1
1	Accept	
2	S3 S2	4
3	r ₂ r ₂	
4	S5	
5	r ₁ r ₁	

Parsing Input String

Input String :- (a)\$ parse using SLR(1) parse Table

Steps	Parsing Stack	S/P String	Action
1.	\$0	(a)\$	Shift 2
2.	\$0(2	a)\$	Shift 3
3.	\$0(2a3	)\$	Reduce R2 A → a ... (2)
4.	\$0(2A \$0(2A4	)\$	Shift 5
5.	\$0(2A4)5	\$	Reduce R1 A → (A) ... (1)
6.	\$0A \$0A1	\$	Accepted.

Date :
Page No.



Scanned with
CamScanner

Syntax-Directed Translation

- Grammar symbols are associated with **attributes** to associate information with the programming language constructs that they represent.
- Values of these attributes are evaluated by the **semantic rules** associated with the production rules.
- Evaluation of these semantic rules:
 - may generate intermediate codes
 - may put information into the symbol table
 - may perform type checking

- When we associate semantic rules with productions, we use two notations:
 - **Syntax-Directed Definitions**
 - **Translation Schemes**
- **Syntax-Directed Definitions:**
 - give high-level specifications for translations
 - hide many implementation details such as order of evaluation of semantic actions.
 - We associate a production rule with a set of semantic actions, and we do not say when they will be evaluated.

- **Translation Schemes:**
 - indicate the order of evaluation of semantic actions associated with a production rule.
 - In other words, translation schemes give a little bit information about implementation details.

- Attribute grammar is a special form of context-free grammar where some additional information (attributes) are appended to one or more of its non-terminals in order to provide context-sensitive information.
- Each attribute has well-defined domain of values, such as integer, float, character, string, and expressions.
- $E \rightarrow E + T \{ E.value = E.value + T.value \}$
- Semantic attributes may be assigned to their values from their domain at the time of parsing and evaluated at the time of assignment or conditions.
- Based on the way the attributes get their values, they can be broadly divided into two categories : synthesized attributes and inherited attributes.

Synthesized attributes

- These attributes get values from the attribute values of their child nodes.
- Each of the non-terminals has a single synthesized attribute, called val.
- An SDD that involves only synthesized attributes is called **S-attributed**
- Each rule computes an attribute for the non-terminal at the head of a production from attributes taken from the body of the production.

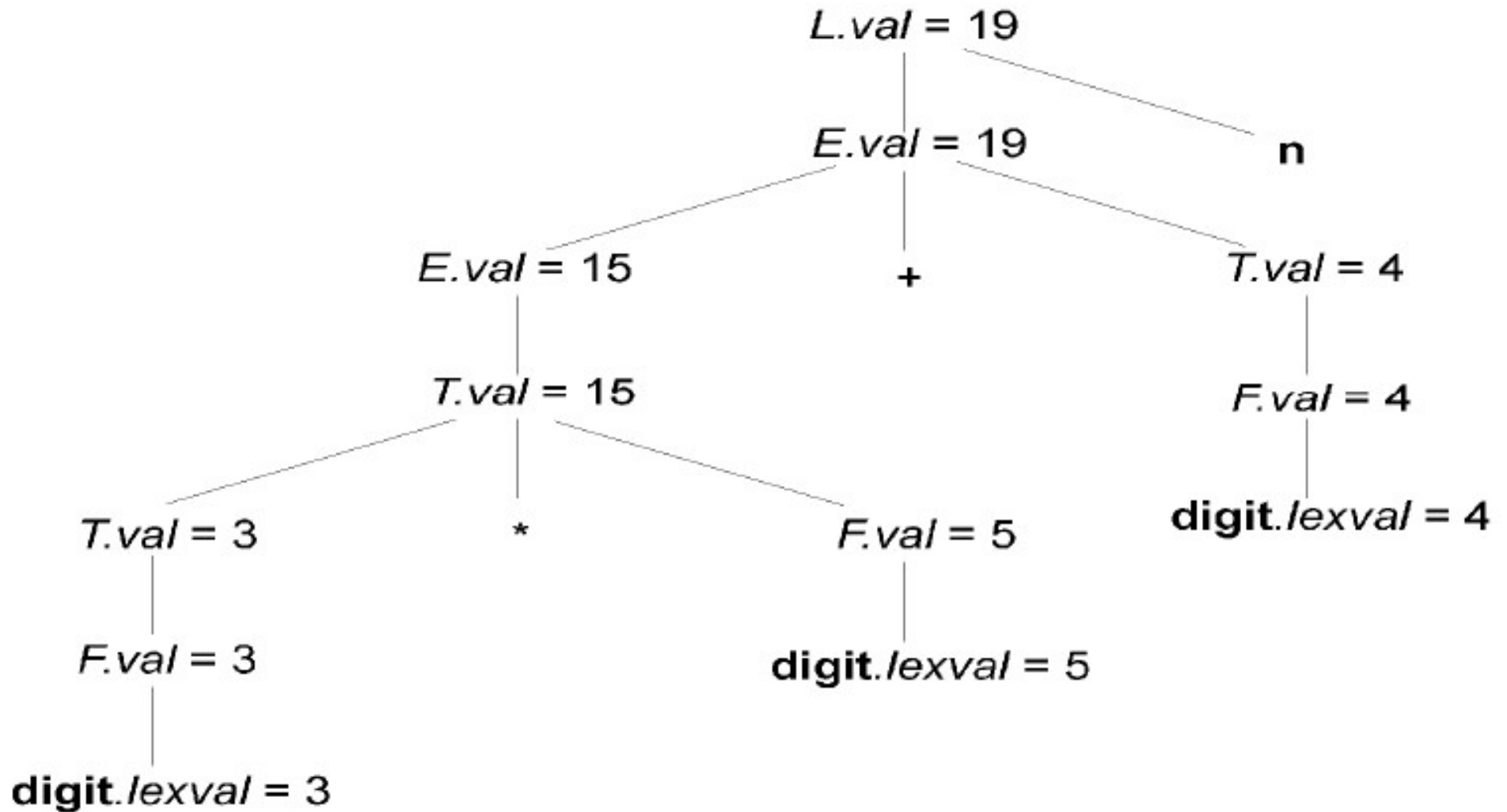
Synthesized attributes

Production	Semantic Rules
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

- A parse tree, showing the value(s) of its attribute(s) is called an **annotated parse tree**.
- With synthesized attributes, evaluate attributes in bottom-up order.

Synthesized attributes

Annotated Parse Tree of $3*5+4n$

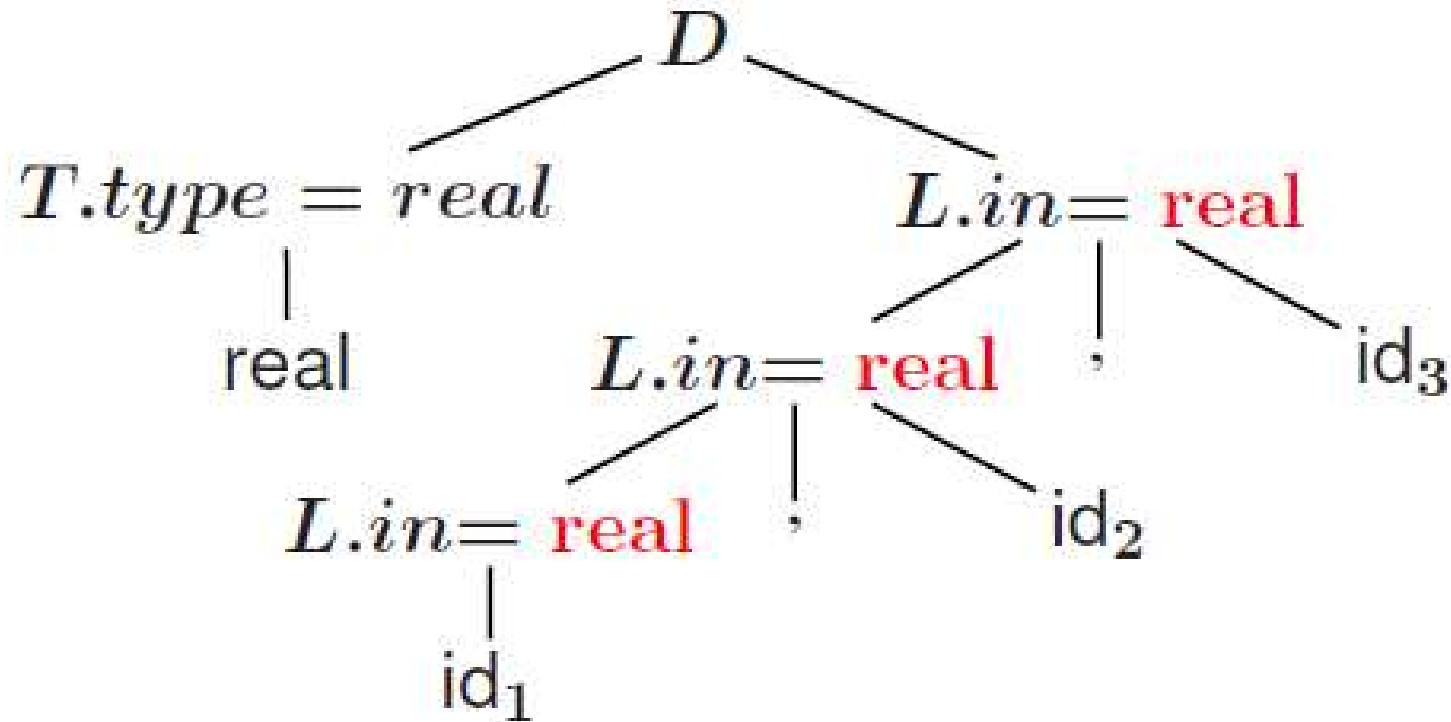


Inherited attributes

PRODUCTION	SEMANTIC RULE
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{real}$	$T.type := \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in := L.in; \text{ addtype}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{ addtype}(\text{id.entry}, L.in)$

Inherited attributes

Annotated parse tree of real id, id, id



1. The combination of grammar and semantic rules are called as:

- ✓ Syntax directed translation
- ✓ semantic directed translation
- ✓ Syntax translator
- ✓ None of these

2. In syntax directed translation, along with the grammar we associate some informal notation are called as:

- ✓ Semantic Rules
- ✓ Syntax Rules
- ✓ parsing Rules
- ✓ None of these

- Which mapping is described by the implementation of the syntax-directed translator?
 - ✓ Parse table
 - ✓ Input
 - ✓ Output
 - ✓ Input-Output

Recap

- An SDD that involves only synthesized attributes is called **S-attributed**
- An SDD that involves only inherit attributes is called **L-attributed**
- A parse tree, showing the value(s) of its attribute(s) is called an **annotated parse tree**

Topic Objective

1. Introduce students to the concepts of syntax directed translation
2. Example of evaluating the SDT with the help of attributes

Implementation of desk calculator

Production

- $S \rightarrow E\$$
- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow I$
- $I \rightarrow I.\text{digit}$
- $I \rightarrow \text{digit}$

Semantic Actions

$\{\text{print val}[\text{Top}]\}$

$\{\text{val}[\text{Top}]\} = \text{val}[\text{Top}-1] + \text{val}[\text{Top}]\}$

$\{\text{val}[\text{Top}]\} = \text{val}[\text{Top}-1] * \text{val}[\text{Top}]\}$

$\{ \}$

$\{\text{val}[\text{Top}]\} = \text{val}[\text{Top}-1] * 10 + \text{lex.val}\}$

$\{\text{val}[\text{Top}]\} = \text{val}[\text{Top}] = \text{lex.val}\}$

Implementation of desk calculator

- Design and implement a desk calculator that can perform basic arithmetic operations (+, -, *, /) and handle parentheses for precedence.
- **Solution Approach:**
- **Understanding Requirements:**
 - The calculator should handle basic arithmetic operations: addition (+), subtraction (-), multiplication (*), and division (/).
 - It should respect operator precedence, where multiplication and division take precedence over addition and subtraction.
 - Parentheses should be used to enforce precedence.
- **Example Calculation:** Consider the expression: $2+3*(4-1)2 + 3 * (4 - 1)2+3*(4-1)$.
- **Step-by-Step Evaluation:**
 - Evaluate inside the parentheses first: $4-1=3$ $4 - 1 = 3$ $4-1=3$.
 - Substitute back into the expression: $2+3*32 + 3 * 32+3*3$.
 - Perform multiplication first: $3*3=9$ $3 * 3 = 9$ $3*3=9$.
 - Add the result to 2: $2+9=11$ $2 + 9 = 11$ $2+9=11$.
- **Execution Strategy:**
 - Start from left to right, respecting parentheses and operator precedence.
 - Use a separate line or space for each step of calculation to maintain clarity.
- **Handling Errors:**
 - Ensure division by zero is not attempted.
 - Recognize and correct input errors like missing operators or parentheses.

Implementation of desk calculator

- **Example Calculation:**
- Given the expression $2+3*(4-1)$, follow these steps:
- Compute $4-1$
- $4-1=3$
- Substitute into the original expression:
- $2+3*3$
- Perform multiplication:
- $3*3=9$
- Add to the initial number:
- $2+9=11$

Recap

- Implementation of desk Calculator

Topic Objective

1. Introduce students to the concepts of intermediate code generation
2. Very useful in the concept of how high level program is stored in address form

Intermediate Code representation

- Front end of a compiler is efficient and can be automated
- Back end is generally hard to automate and finding the optimum solution requires exponential time
- Intermediate code generation can affect the performance of the back end
- Goal: Generate a Machine Independent Intermediate Form that is Suitable for Optimization and Portability
- Facilitates retargeting: enables attaching a back end for the new machine to an existing front end
- Enables machine-independent code optimization

Intermediate Code representation

- Graphical representations
- e.g. Abstract Syntax Tree
- Postfix notation: operations on values stored on operand stack
- Three-address code: (e.g. triples and quads)

$x := y \text{ op } z$

- Two-address code:

$x := \text{op } y$

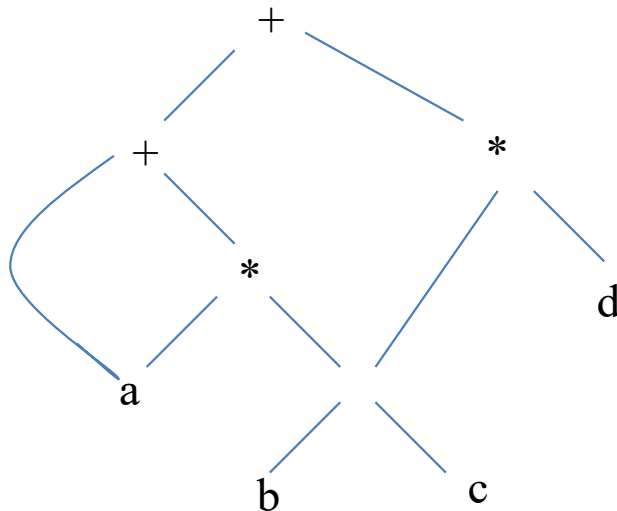
- A parse tree, also known as a concrete syntax tree, represents the syntactic structure of a string according to the rules of a formal grammar. It's constructed during the parsing process of a compiler or interpreter. Parse trees are hierarchical structures where nodes represent terminals (tokens) and non-terminals (grammar rules). Here's an example to illustrate:
- **Example of a Parse Tree**
- Consider the grammar and input expression:
- Grammar:
 - $E \rightarrow E + E$
 - $E \rightarrow E * E$
 - $E \rightarrow (E)$
 - $E \rightarrow \text{num}$
- Input Expression: $2+3*(4-1)$
- $+$
- $/\backslash$
- $2 \quad *$
- $/\backslash$
- $3 \quad -$
- $/\backslash$
- $4 \quad 1$

Syntax Tree (Abstract Syntax Tree, AST)

- A syntax tree, or abstract syntax tree (AST), represents the syntactic structure of a program or expression after the parsing phase. Unlike parse trees, syntax trees abstract away from specific syntactic details and focus on the program's logical structure. They are used for semantic analysis, optimization, and code generation phases in compilers and interpreters. Here's how it differs from a parse tree:
- **Example of a Parse Tree**
- Consider the grammar and input expression:
- Grammar:
 - $E \rightarrow E + E$
 - $E \rightarrow E * E$
 - $E \rightarrow (E)$
 - $E \rightarrow \text{num}$
- Input Expression: $2+3*(4-1)$
- $+$
- $/\backslash$
- $2 \quad *$
- $/\backslash$
- $3 \quad -$
- $/\backslash$
- $4 \quad 1$

Three address code(CO3)

- In a three address code there is at most one operator at the right side of an instruction
- Example:



$t1 = b - c$
 $t2 = a * t1$
 $t3 = a + t2$
 $t4 = t1 * d$
 $t5 = t3 + t4$

- Quadruples
 - Has four fields: op, arg1, arg2 and result
- Triples
 - Temporaries are not used and instead references to instructions are made
- Indirect triples
 - In addition to triples we use a list of pointers to triples

Three address codes Example(CO3)

Three address code

- Example
- $b * \text{minus } c + b * \text{minus } c$

$t1 = \text{minus } c$

$t2 = b * t1$

$t3 = \text{minus } c$

$t4 = b * t3$

$t5 = t2 + t4$

$a = t5$

Quadruples

op	arg1	arg2	result
minus	c		t1
*	b	t1	t2
minus	c		t3
*	b	t3	t4
+	t2	t4	t5
=	t5		a

Triples

	op	arg1	arg2
0	minus	c	
1	*	b	0
2	minus	c	
3	*	b	2
4	+	1	3
5	=	a	4

Indirect Triples

	op		op	arg1	arg2
0	0	0	minus	c	
1	1	1	*	b	0
2	2	2	minus	c	
3	3	3	*	b	2
4	4	4	+	1	3
5	5	5	=	a	4

Quadruples Example(CO3)

$a := b * -c + b * -c$

	op	arg1	arg2	result
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		a

Triples Example(CO3)

$a := b * -c + b * -c$

	op	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	:=	a	(4)

Result is implicit in triples

Indirect triples(CO3)

$a := b * -c + b * -c$

	<i>op</i>		<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	(14)	(14)	uminus	c	
(1)	(15)	(15)	*	b	(14)
(2)	(16)	(16)	uminus	c	
(3)	(17)	(17)	*	b	(16)
(4)	(18)	(18)	+	(15)	(17)
(5)	(19)	(19)	assign	a	(18)

- Three address code statement are.....
 - ✓ Assignment Statement
 - ✓ Copy Statement
 - ✓ procedure call
 - ✓ All of the above

- Postfix notation of $a + b * c$ is ____ .
 - ✓ $abc*+$
 - ✓ $abc+*$
 - ✓ $ba+c*$
 - ✓ None of the above

- A parse tree showing the value of attributes at each node is called-----
 - ✓ Syntax tree
 - ✓ Annotated parse tree
 - ✓ parse tree
 - ✓ All of the above.

CFG is used to formally define the structure of valid sentences in a language.

: In a Voice Assistant:

- CFG provides **rules** that describe how words can be combined.
- It helps the system understand **syntax (sentence structure)**.
- Ensures that user input follows recognizable patterns.

Example:

A simple CFG rule:

$S \rightarrow VP$

$VP \rightarrow \text{Verb Object}$

$\text{Verb} \rightarrow \text{"play"} \mid \text{"open"}$

$\text{Object} \rightarrow \text{"music"} \mid \text{"YouTube"}$

If a user says:

"play music" → It matches the CFG rules → valid command.

If user says:

"music play quickly" → May not match → needs correction or advanced parsing.

Role of Parsing

Parsing is the process of analyzing the sentence using CFG rules.

In a Voice Assistant:

Converts spoken input → structured representation (parse tree)

Identifies:

Intent (what user wants)

Entities (objects involved)

Example:

User input: "**Play music**"

Parsing result:

Verb → *play*

Object → *music*

👉 System interprets:

Intent = Play

Entity = Music

Role of CFG (Context-Free Grammar)

- **Defines sentence structure**
- CFG specifies how words can be arranged to form valid commands.
- **Limits possible inputs**
- Helps the system focus only on expected patterns (e.g., “play music”, “open app”).
-
- **Provides grammatical rules**
- Uses production rules (like $S \rightarrow \text{Verb} + \text{Object}$) to represent language.
- **Improves understanding accuracy**
- Reduces incorrect interpretations by filtering invalid sentences.
- **Useful for command-based systems**
- Works well where user input follows fixed formats.

- **Analyzes user input**
- Checks whether the spoken sentence follows CFG rules.
-
- **Builds structure (Parse Tree)**
- Breaks the sentence into components like verb, object, etc.
-
- **Extracts meaning (Intent & Entities)**
- Identifies what action is needed and on what object.
-
- **Detects errors or ambiguity**
- Helps identify if the sentence is incomplete or unclear.
-
- **Converts text into machine-understandable form**
- Prepares structured data for further processing.

Combined Role (CFG + Parsing)

- CFG provides **rules**, parsing applies **those rules**
- Together, they help the system **understand commands correctly**
 -
- Enable **accurate execution of user requests**
- Form the **basic language processing layer** in voice assistants

Role of CFG and Parsing in Voice Assistant

A voice assistant converts spoken language into meaningful actions. **Context-Free Grammar (CFG)** and **Parsing** play an important role in understanding the structure and meaning of user commands.

Faculty Video Links, Youtube & NPTEL Video Links and Online Courses Details

Youtube/other Video Links

- <https://www.youtube.com/watch?v=GOlsYofJjyQ>
- https://www.youtube.com/watch?v=sMxqUQc_jHQ&t=360s
- <https://nptel.ac.in/courses/106/105/106105190/>
- <https://nptel.ac.in/courses/106/105/106105190/>

- Calculate the first and follow function of production rules:

$$S \rightarrow ACB / CbB / Ba$$

$$A \rightarrow da / BC$$

$$B \rightarrow g / \epsilon$$

$$C \rightarrow h / \epsilon$$

- Consider the following grammar and eliminate left recursion-

$$E \rightarrow E + E / E \times E / a$$

- Do left factoring in the following grammar-

$$S \rightarrow bSSaaS / bSSaSb / bSb / a$$

- Consider this following grammar:

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

Check whether it is LL(1) grammar or not.

- Consider the following grammar-

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L , S \mid S$$

Parse the input string $(a , (a , a))$ using a shift-reduce parser.

- Consider the grammar

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

Perform SLR(1) and LR (0) parsing on above grammar.

➤ Consider the grammar and find the grammar is -

$$E \rightarrow A \mid B$$

$$A \rightarrow a \mid c$$

$$B \rightarrow b \mid c$$

- a) LR(0)
- b) LR(0) & LR(1)
- c) LR(1), SLR(1)
- d) none

➤ Consider the grammar $E \rightarrow E + n \mid E \times n \mid n$. For a sentence " $n + n \times n$," the handles in the right sentential form of reduction are

- a) n , $E + n$ and $E + n \times n$
- b) n , $E + n$ and $E + E \times n$
- c) n , $n + n$ and $n + n \times n$
- d) n , $E + n$ and $E \times n$**

➤ Give the Follow set for the non-terminal E in the grammar.

$E' \rightarrow E$

$E \rightarrow \text{num}$

$E \rightarrow (+A)$

$A \rightarrow AA$

$A \rightarrow E$

a) $\{\text{num}, (,)\}$

b) $\{ (,), \text{num}, \$ \}$

c) $\{ (, \text{num}, \$ \}$

d) $\{), \text{num}, \$ \}$

➤ Eliminate left recursion from the following grammar:

$$A \rightarrow Bd \mid Aff \mid CB$$

$$B \rightarrow Bd \mid Bf \mid df$$

$$C \rightarrow h \mid g$$

a. $A \rightarrow Bd AI \mid CB AI$

$$AI \rightarrow ff AI \mid \epsilon$$

$$B \rightarrow df BI$$

$$BI \rightarrow d BI \mid fBI \mid \epsilon$$

$$C \rightarrow h \mid g$$

b. $A \rightarrow Bd AI \mid CB AI$

$$A \rightarrow ff AI \mid \epsilon$$

$$B \rightarrow df BI$$

$$BI \rightarrow d BI \mid fBI \mid \epsilon$$

$$C \rightarrow h \mid g$$

c. $A \rightarrow Bd AI \mid CB AI$

$$AI \rightarrow ff AI$$

$$B \rightarrow df BI$$

$$BI \rightarrow d BI \mid fBI$$

$$C \rightarrow h \mid g$$

d. $A \rightarrow Bd AI \mid CB$

$$AI \rightarrow ff AI \mid \epsilon$$

$$B \rightarrow df BI$$

$$BI \rightarrow d BI \mid fBI \mid \epsilon$$

$$C \rightarrow h \mid g$$

➤ Consider the grammar shown below

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

In LL(1) table M of this grammar, the entries $M[S', e]$ and $M[S', \$]$ are

- a) $S' \rightarrow eS$, $S' \rightarrow \varepsilon$
- b) $S' \rightarrow eS$, $\{\}$
- c) $S' \rightarrow \varepsilon$, $S' \rightarrow \varepsilon$
- d) $\{S' \rightarrow eS \ \& \ S' \rightarrow \varepsilon\}$, $S' \rightarrow \varepsilon$

➤ Give the Follow of B in the following grammar:

$$A \rightarrow B C D$$

$$B \rightarrow w \mid B x$$

$$C \rightarrow y C z \mid m$$

$$D \rightarrow D B \mid a$$

- a) $\{x, y, m, \$\}$
- b) $\{x, y, w, \$\}$
- c) $\{x, w, m, \$\}$
- d) $\{x, y, m, w, \$\}$

- Using LR(0) items we can construct.
 - a) SLR parsing table
 - b) LALR parsing table
 - c) CALR parsing table
 - d) None
- Which of the following is true?
 - a) every SLR(1) is LR(0)
 - b) every LL(1) is LR(1)
 - c) every LL(1) is SLR(1)
 - d) every LL(1) is LR(0)
- If a grammar is SLR(1) then []
 - a) it may have S/R conflict
 - b) It will not have any conflict
 - c) It may have R/R conflict
 - d) it will not have S/R but may have R/R conflict

- Consider the grammar $S \rightarrow (S) \mid a$. If SLR(1) has n_1 states, LR(1) has n_2 states, LALR(1) has n_3 states, which of the following is true?
- a) $n_1 < n_2 < n_3$
 - b) $n_1 = n_3 < n_2$
 - c) $n_1 = n_2 = n_3$
 - d) $n_1 > n_3 > n_2$

1. Consider the following grammar

$$S \rightarrow AS \mid b$$

$$A \rightarrow SA \mid a$$

Show that the grammar is ambiguous.

2. Find LR(0) Canonical collections.

3. Construct the SLR parsing table.

Weekly Assignments-2

1. From a given grammar remove left RECURSION $E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$

2. Parse the strings “ab” , “acdb” and “adb” using stack.

$$S \rightarrow aABb$$

$$A \rightarrow c \mid \epsilon$$

$$B \rightarrow d \mid \epsilon$$

3. Parse the string “ (id*id) ” and “ id+id+id ” using stack.

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

➤ Handle pruning forms the basis of _____ .

(a) Bottom-up parsing

(b) Top-down parsing

(c) Both (a) and (b)

(d) None of these

➤ Non-recursive predictive parser contains

_____ .

(a) An input buffer

(b) A parsing table

(c) An output stream

(d) All of these

➤ If $A \rightarrow \alpha\beta$ is a production and a is terminal symbol or right end marker $\$,$ then LR(1) items will be defined by the production _____.

(a) $[A \rightarrow \alpha.\beta, a]$

(b) $[A \rightarrow .\alpha\beta, a]$

(c) $[A \rightarrow \alpha.\beta a]$

(d) $[A \rightarrow a.\beta, \alpha]$

➤ Consider the grammar shown below

$$S \rightarrow FR, R \rightarrow *S \mid \epsilon, F \rightarrow id$$

In LL (1) table M of this grammar, the entries $M[S, id]$ and $M[R, \$]$ are

a) $S \rightarrow FR, R \rightarrow \epsilon$

b) $S \rightarrow FR \ \& \ \{\}$

c) $S \rightarrow FR \ \& \ R \rightarrow *S$

d) $F \rightarrow id, R \rightarrow \epsilon$

➤ Top-down parsing is a technique to find _____ .

- (a) Leftmost derivation
- (b) Rightmost derivation
- (c) Leftmost derivation in reverse
- (d) Rightmost derivation in reverse

➤ Predictive parsing is possible only for _____ .

- (a) LR(k) grammar
- (b) LALR(1) grammar
- (c) LL(k) grammar
- (d) CLR(1) grammar

- Which two functions are required to construct a parsing table in predictive parsing technique?
- (a) CLOSURE() and GOTO ()
 - (b) FIRST() and FOLLOW()
 - (c) ACTION() and GOTO()
 - (d) None of these
- Which of these parsing techniques is a kind of bottom-up parsing?
- (a) Shift-reduce parsing
 - (b) Reduce-reduce parsing
 - (c) Predictive parsing
 - (d) Recursive-decent parsing

- Which of the following methods is used by the bottom-up parser to generate a parse tree?
- (a) Leftmost derivation
 - (b) Rightmost derivation
 - (c) Leftmost derivation in reverse
 - (d) Rightmost derivation in reverse
- Handle pruning forms the basis of _____ .
- (a) Bottom-up parsing
 - (b) Top-down parsing
 - (c) Both (a) and (b)
 - (d) None of these

Glossary Questions

1. Explain the parsing techniques with a hierarchical diagram.
2. What are the problems associated with Top Down Parsing?
3. Write the production rules to eliminate the left recursion and left factoring problems.
4. Consider the following Grammar:
 $A \rightarrow ABd|Aa|a$ $B \rightarrow Be|b$
Remove left recursion.
5. Do left factoring in the following grammar: $A \rightarrow aAB|aA|a$ $B \rightarrow bB|b$
6. Write a short note on:
 - a. Ambiguity (with example)
 - b. Recursive Descent Parser
 - c. Predictive LL(1) parser (working)
 - d. Handle pruning
 - e. Operator Precedence Parser
7. Write Rules to construct FIRST Function and FOLLOW Function.

- Listing pointers to triples, rather than listing triples themselves is implementation of
 - a. Quadruple
 - b. Triple
 - c. Indirect Triple
 - d. Three address code
- Quadruple is a record structure with four fields.
 - a. Op, arg1, arg2 and result
 - b. op1, op2, agr2 and result
 - c. agr1, arg2 , result and op
 - d. All of d above

Old Question Paper

Printed Page:- 06

Subject Code:- ACSE0504/ACSEH0504

Roll. No:

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA
(An Autonomous Institute Affiliated to AKTU, Lucknow)

B.Tech.

SEM: V - THEORY EXAMINATION DEC-2023

Subject: Compiler Design

Time: 3 Hours

Max. Marks: 100

General Instructions:

IMP: Verify that you have received the question paper with the correct course, code, branch etc.

1. This Question paper comprises of three Sections -A, B, & C. It consists of Multiple Choice Questions (MCQ's) & Subjective type questions.
2. Maximum marks for each question are indicated on right -hand side of each question.
3. Illustrate your answers with neat sketches wherever necessary.
4. Assume suitable data if necessary.
5. Preferably, write the answers in sequential order.
6. No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.

SECTION A

20

1. Attempt all parts:-

- 1-a. Lexical analysis is about breaking a sequence of characters into 1
(CO1)
- (a) Tokens
 - (b) Lines
 - (c) Groups
 - (d) Packets
- 1-b. _____ or scanning is the process, where the stream of characters making up the source program is read from left to right and grouped into tokens. 1
(CO1)
- (a) Modeling
 - (b) Diversion
 - (c) Lexical analysis
 - (d) All of these

Page 1 of 6

- 1-c. LL(1) parsing table is constructed between _____. (CO2) 1
 (a) terminals and terminals
 (b) terminals and non-terminals
 (c) non-terminals and non-terminals
 (d) none of the above
- 1-d. In Top-Down parsing, Root node of parse tree represents with _____. (CO2) 1
 (a) terminals
 (b) Start Symbol
 (c) operators
 (d) none of above
- 1-e. An SDD that involves only synthesized attributes is called _____. (CO3) 1
 (a) S-attributed
 (b) L-attributed
 (c) S and L attributed
 (d) None of the above.
- 1-f. Intermediate code representation form are _____. (CO3) 1
 (a) Syntax tree
 (b) Postfix notation
 (c) Three address code
 (d) all of these
- 1-g. Who is responsible for the creation of the symbol table? (CO4) 1
 (a) Assembler
 (b) Compiler
 (c) Interpreter
 (d) None of above
- 1-h. Symbol table is used by which phase of compiler. (CO4) 1
 (a) Lexical analysis
 (b) Syntax analysis
 (c) Semantic analysis
 (d) All of above
- 1-i. _____ is a tool that depicts the structure of basic blocks, helps to see the 1

flow of values flowing among the basic blocks, and offers optimization too.
(CO5)

- (a) DAG
- (b) CAG
- (c) SAG
- (d) PAG

- 1-j. In Directed Acyclic Graph, Leaf nodes represented by _____. (CO5) 1
- (a) identifiers
 - (b) names
 - (c) constants
 - (d) All of the above

2. Attempt all parts:-

- 2.a. Differentiate between analysis and synthesis phase. (CO1) 2
- 2.b. Define ambiguous grammar. (CO2) 2
- 2.c. Define Syntax directed translation. (CO3) 2
- 2.d. Define activation record. (CO4) 2
- 2.e. What are the rules to determine the leaders of basic blocks? (CO5) 2

SECTION B

30

3. Answer any five of the following:-

- 3-a. Define term "pass" in the compiler and Differentiate between single pass and multi pass compiler. Also Discuss the advantages and disadvantages of single and multipass compiler. (CO1) 6
- 3-b. Define the term lexeme, token, Pattern with examples. 6
Consider the following statement in c programming-
printf(" i= %d,&i= %x",i,&i);
find all the tokens generated by lexical analyzer. (CO1)
- 3-c. Consider the grammar: 6
S → E
E → E+E / E*E
E → num
E → id
Input to parse as **id+num*id**. Perform shift reduce parsing. (CO2)
- 3-d. Write a short note on: (CO2) 6

- a) Recursive Descent Parser,
b) Operator Precedence Parser
- 3.e. Write Short note on: (CO3) 6
a) Parse Tree
b) Annotated Parse Tree
c) Syntax Tree
- 3.f. Explain Symbol table with various operations in details. (CO4) 6
- 3.g. Write down the Algorithm for construction of Directed Acyclic Graph. (CO5) 6
Construct DAG for given Expression.
T1 = a + b
T2 = a - b
T3 = T1 * T2
T4 = T1 - T3
T5 = T4 + T3

SECTION C

50

4. Answer any one of the following:-

- 4-a. Discuss the various phases of compiler and trace the program segment P: =I + 10
R *20. for all phases. (CO1)
- 4-b. Discuss the following terms: (CO1) 10
a) Bootstrapping
b) Cross Compiler
c) YACC
d) LEX

5. Answer any one of the following:-

- 5-a. Define bottom up parser and its types in brief. (CO2) 10
Construct an SLR parsing table for the given grammar & parse input string
id*id+id.
E → E+T
E → T
T → T*F
T → F
F → (E)
F → id
- 5-b. Write the algorithm for FIRST and FOLLOW. Also Find FIRST and FOLLOW sets 10
of given production rules (CO2)

$S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bC / \epsilon$
 $D \rightarrow EF$
 $E \rightarrow g / \epsilon$
 $F \rightarrow f / \epsilon$

6. Answer any one of the following:-

6-a. Define Three Address Code. also Discuss the representations of three address code. (CO3) 10

Write the quadruples, triples, Indirect Triples for the following expression
 $(x + y) * (y + z) + (x + y + z)$

6-b. Define the Intermediate Code. Write down the Types of Three Address Statements. (CO3) 10

Generate the three-address code for the following program segment where, x and y are arrays of size 10 * 10, and there are 4 bytes/word.

```

begin
add = 0
a = 1
b = 1
do
begin
add = add + x[a,b] * y[a,b]
a = a + 1
b = b + 1
end
while a <= 10 and b <= 10
end
    
```

7. Answer any one of the following:-

7-a. Define Error handler and identify the various types of errors and recovery methods. (CO4) 10

7-b. Explain the different Data Structures that can be used to implement Symbol tables.(CO4) 10

8. Answer any one of the following:-

8-a. Define basic block and flow graph. Write the step to partition a sequence of 3 address statements into basic blocks. (CO5) 10

Generate the flow-graphs for the following expressions
 1. PROD = 0

Old Question Paper

```
2. I = 1
3. T2 = addr(A) - 4
4. T4 = addr(B) - 4
5. T1 = 4 x I
6. T3 = T2[T1]
7. T5 = T4[T1]
8. T6 = T3 x T5
9. PROD = PROD + T6
10. I = I + 1
11. IF I <= 20 GOTO (5)
```

8-b. Explain purpose of code optimization? Explain in detail about various loop optimization techniques with example. (CO5) 10

NIET

Old Question Paper

Printed Page:-

Subject Code:- ACSE0504

Roll. No:

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA

(An Autonomous Institute Affiliated to AKTU, Lucknow)

B.Tech.

SEM: V - THEORY EXAMINATION (2022 - 2023)

Subject: Compiler Design

Time: 3 Hours

Max. Marks: 100

General Instructions:

IMP: Verify that you have received the question paper with the correct course, code, branch etc.

1. This Question paper comprises of three Sections -A, B, & C. It consists of Multiple Choice Questions (MCQ's) & Subjective type questions.
2. Maximum marks for each question are indicated on right -hand side of each question.
3. Illustrate your answers with neat sketches wherever necessary.
4. Assume suitable data if necessary.
5. Preferably, write the answers in sequential order.
6. No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.

SECTION A

20

1. Attempt all parts:-

- 1-a. The lexical analyzer takes ____ as input and produces a list of ____ as output. (CO1) 1
- (a) Machine code, mnemonic
 - (b) Tokens, source code
 - (c) Source code, tokens
 - (d) Both Machine code, mnemonic and Tokens, source code
- 1-b. A compiler for a high level language that runs on one machine and produce code for different machine is called _____. (CO1) 1
- (a) One pass compiler
 - (b) multipass compiler
 - (c) croos compiler
 - (d) optimizing compiler
- 1-c. Parsing is categorized into how many types?(CO2) 1
- (a) 2
 - (b) 3

Old Question Paper

- (c) 1
(d) 4
- 1-d. CLR parsing is a type of _____. (CO2) 1
 (a) Bottom-Up
 (b) Top-Down
 (c) Both Bottom-Up and Top-Down
 (d) None of above
- 1-e. Type checking is normally done during _____. (CO3) 1
 (a) Lexical analysis
 (b) Syntax Analysis
 (c) Syntax directed translation
 (d) Code optimization
- 1-f. A parse tree showing the value of attributes at each node is called _____. (CO3) 1
 (a) Syntax tree
 (b) Annotated parse tree
 (c) parse tree
 (d) All of the above.
- 1-g. Activation Record is a _____. (CO4) 1
 (a) Attribute
 (b) Non-Contiguous block of memory
 (c) Contiguous block of memory
 (d) None of above
- 1-h. Missing parenthesis is a _____. (CO4) 1
 (a) Semantic error
 (b) Syntax error
 (c) Both Semantic Error and Syntax Error
 (d) None of above
- 1-i. Dead code plays no role in any program operation and therefore it can simply be eliminate (CO5) 1
 (a) TRUE
 (b) FALSE

- (c) Can be true or false
(d) Can not say
- 1-j. Code generation can be considered as the _____. (CO5) 1
- (a) first phase of compilation
(b) second phase of compilation
(c) third phase of compilation
(d) final phase of compilation
2. Attempt all parts:-
- 2.a. Differentiate between Token, Pattern, and Lexeme. (CO1) 2
- 2.b. What is an operator precedence parser? (CO2) 2
- 2.c. Explain S- attribute and L- attribute. (CO3) 2
- 2.d. What are the goals of Error Handler? (CO4) 2
- 2.e. Define DAG with examples. (CO5) 2
- SECTION B** 30
3. Answer any five of the following:-
- 3-a. How to solve the source program to target machine code by using language processing system? (CO1) 6
- 3-b. Classify the concepts of compiler and Interpreter. (CO1) 6
- 3-c. Perform LL(1) parsing to parse string "a+a" by using the following grammar : (CO2) 6
- S → F
S → S + F
F → a
- 3-d. Parse string "ccdd" using given grammar with the help of shift-reduce parsing : (CO2) 6
- S → CC
C → cC
C → d
- 3.e. Consider the following SDTS. (CO3) 6
- E → E + T { E.val = E.val + T.val }
E → T { E.val = T.val }
T → T * F { T.val = T.val * F.val }
T → F { T.val = F.val }
F → INTLIT { F.val = INTLIT.lexval }
- Using above SDTS, Construct parse tree and annotated parse tree for the given expression "

- 2+3*4 " .
- 3.f. Draw the format of Activation Record in stack allocation and explain each field in it (CO4) 6
- 3.g. Explain Peephole optimization with various techniques. (CO5) 6
- SECTION C** 50
4. Answer any one of the following:-
- 4-a. Obtain Leftmost and Rightmost derivations for the input string $a+a*a$ using following Production rules- (CO1) 10
- $X \rightarrow X + X$
 $X \rightarrow X * X$
 $X \rightarrow a$
- Also draw Parse Tree for the same.
- 4-b. Define the term "Compiler". Explain the different phases of compiler in detail. Write down the output of each phase for the expression $a:=b+c*50$. (CO1) 10
5. Answer any one of the following:-
- 5-a. Construct CLR parsing table for the given grammar (CO2) 10
- $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$
- Convert that table into LALR parsing table.
- 5-b. Write the algorithm for FIRST and FOLLOW. Also Find FIRST and FOLLOW sets of given production rules (CO2) 10
- $S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bC / \epsilon$
 $D \rightarrow EF$
 $E \rightarrow g / \epsilon$
 $F \rightarrow f / \epsilon$
6. Answer any one of the following:-
- 6-a. Explain the syntax directed translation scheme and evaluate the expression $(3*5+4)$ using simple Desk calculator. (CO3) 10
- 6-b. Compare between Quadruples , Triples and Indirect Triples. Also Translate the expression $(p + q)*(q + r) + (p + q + r)$ into Quadruples , Triples and Indirect Triples representation. (CO3) 10
7. Answer any one of the following:-

- 7-a. Define Errors and its types with their Error Recovery methods . Explain with example. (CO4) 10
- 7-b. Explain the different Data Structures that can be used to implement Symbol tables.(CO4) 10
8. Answer any one of the following:-
- 8-a. Define the terms basic blocks, flow graphs and loop in flow graph. Generate the basic block and flow-graphs for the following expressions - (CO5) 10
- 1) $r = 1$
 - 2) $c = 1$
 - 3) $t1 = 10 * r$
 - 4) $t2 = t1 + c$
 - 5) $t3 = 8 * t2$
 - 6) $t4 = t3 - 88$
 - 7) $a[t4] = 0.0$
 - 8) $c = c + 1$
 - 9) if $c \leq 10$ goto (3)
 - 10) $r = r + 1$
 - 11) if $r \leq 10$ goto (2)
 - 12) $r = 1$
 - 13) $t5 = c - 1$
 - 14) $t6 = 88 * t5$
 - 15) $a[t6] = 1.0$
 - 16) $r = r + 1$
 - 17) if $r \leq 10$ goto (13)
- 8-b. What is loop optimization? Explain in details about various loop optimization techniques with proper examples.. (CO5) 10

Printed Pages:2

Sub Code:RCS602

Paper Id: **110263**

Roll No.

--	--	--	--	--	--	--	--	--	--

BTECH
(SEM VI) THEORY EXAMINATION 2018-19
COMPILER DESIGN

Time: 3 Hours

Total Marks: 70

Note: 1. Attempt all Sections. If require any missing data; then choose suitably.

SECTION A

1. Attempt all questions in brief. 2 x 7 = 14

- a. What are the two parts of a compilation? Explain briefly.
- b. What is meant by viable prefixes?
- c. What are the classifications of a compiler?
- d. List the various error recovery strategies for a lexical analysis.
- e. What is dangling else problem?
- f. What are the various types of intermediate code representation?
- g. Define peephole optimization.

SECTION B

2. Attempt any three of the following: 7 x 3 = 21

- a. Write the quadruples ,triple and indirect triple for the following expression:
 $(x+y)*(y+z)+(x+y+z)$
- b. What are the problems with top down parsing? Write the algorithm for FIRST and FOLLOW.
- c. Perform Shift Reduce Parsing for the given input strings using the grammar

$$S \rightarrow (L)a$$

$$L \rightarrow L, S | S$$
 - i) $(a,(a,a))$
 - ii) (a,a)
- d. What is global data flow analysis? How does it use in code optimization?
- e. Construct LR(0) parsing table for the following grammar

$$S \rightarrow cB \mid ccA$$

$$A \rightarrow cA \mid a$$

$$B \rightarrow ccB \mid b$$

SECTION C

3. Attempt any one part of the following: 7 x 1 = 7

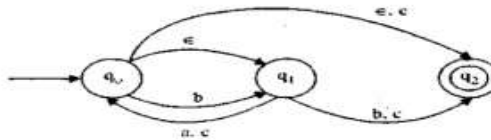
- (a) Convert following NFA to equivalent DFA and hence minimize the number of states in the DFA.

<http://www.aktuonline.com>

www.aktuonline.com

Printed Pages:2

Sub Code:RCS602



(b) Explain the various parameter passing mechanisms of a high level language.

4. Attempt any *one* part of the following: 7 x 1 = 7

(a) How would you represent the following equation using DAG?

$$a := b * -c + b * -c$$

(b) Distinguish between static scope and dynamic scope. Briefly explain access to non-local names in static scope.

5. Attempt any *one* part of the following: 7 x 1 = 7

(a) Write short notes on the following with the help of example:

- (i) Loop unrolling
- (ii) Loop Jamming
- (iii) Dominators
- (iv) Viable Prefix

(b) Draw the format of Activation Record in stack allocation and explain each field in it.

6. Attempt any *one* part of the following: 7 x 1 = 7

(a) Write down the translation procedure for control statement and switch statement

(b) Define Syntax Directed Translation. Construct an annotated parse tree for the expression $(4 * 7 + 1) * 2$, using the simple desk calculator grammar.

7. Attempt any *one* part of the following: 7 x 1 = 7

(a) Explain in detail the error recovery process in operator precedence parsing method.

(b) Explain what constitute a loop in flow graph and how will you do loop optimizations in code optimization of a compiler.

www.aktuonline.com

<http://www.aktuonline.com>

Old Question Paper



PAPER ID-411134

Printed Page: 1 of 2
Subject Code: KCS502

Roll No:

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

**B.TECH.
(SEM V) THEORY EXAMINATION 2021-22
COMPILER DESIGN**

Time: 3 Hours

Total Marks: 100

Note: 1. Attempt all Sections. If require any missing data; then choose suitably.

SECTION A

1. Attempt all questions in brief.

2 x 10 = 20

- a. What is the difference between parse tree and abstract syntax tree?
- b. Explain the problems associated with top-down Parser.
- c. What are the various errors that may appear in compilation process?
- d. What are the two types of attributes that are associated with a grammar symbol?
- e. Define the terms Language Translator and compiler.
- f. What is hashing? Explain.
- g. What is do you mean by left factoring the grammars? Explain.
- h. Define left recursion. Is the following grammar left recursive?
 $E \rightarrow E+E \mid E * E \mid a \mid b$
- i. What is an ambiguous grammar? Give example.
- j. List down the conflicts during shift-reduce parsing.

SECTION B

2. Attempt any three of the following:

10 x 3 = 30

- a. Construct the LALR parsing table for the given grammar
 $S \rightarrow BB$
 $B \rightarrow aB \mid b$
- b. What is an activation record? Explain how it is related with runtime storage organization?
- c. Write the quadruple, triple, indirect triple for the following expression
 $(x + y) * (y + z) + (x + y + z)$
- d. Discuss the following terms:
 - i. Basic block
 - ii. Next use information
 - iii. Flow graph
- e. Construct predictive parse table for the following grammar.
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow F / a \mid b$

SECTION C

3. Attempt any one part of the following:

10 x 1 = 10

- a. Construct the SLR parse table for the following Grammar
 $E \rightarrow E+E$
 $E \rightarrow E * E$
 $E \rightarrow id$
- b. Differentiate between stack allocation and heap allocation.



PAPER ID-411134

Printed Page: 2 of 2
Subject Code: KCS502

Roll No:

--	--	--	--	--	--	--	--	--	--	--	--	--

4. Attempt any *one* part of the following:

10 x 1 = 10

a. Write syntax directed definition for a given assignment statement:

```
S → id=E
E → E+E
E → E*E
E → -E
E → (E)
E → id
```

b. What are the advantages of DAG? Explain the peephole optimization.

5. Attempt any *one* part of the following:

10 x 1 = 10

a. What do you understand by lexical phase error and syntactic error? Also suggest methods for recovery of errors.

b. Discuss how induction variables can be detected and eliminated from the given intermediate code

```
B2: i:= i+1
t1:=4*j
t2:=a[t1]
if t2<10 goto B2
```

6. Attempt any *one* part of the following:

10 x 1 = 10

a. Test whether the grammar is LL(1) or not, and construct parsing table for it.

```
S → IAB / ε
A → IAC / 0C
B → 0S
C → I
```

b. Distinguish between static scope and dynamic scope. Briefly explain access to non local names in static scope.

7. Attempt any *one* part of the following:

10 x 1 = 10

a. What are the various issues in design of code generator & code loop optimization?

b. Generate the three address code for the following code fragment.

```
while(a>b)
{
  if(c<d)
    x=y+z;
  else
    x=y-z;
}
```

Expected Question for University Exam

➤ Consider the following grammar $S \rightarrow AS|b$ $A \rightarrow SA|a$. Construct the SLR parse table for the grammar. Show the actions of the parser for the input string “abab”.

➤ What is a syntax tree? Draw the syntax tree for the following statement: $c \ b \ c \ b \ a \ -$

$* \ + \ - \ * \ =$

➤ Perform Shift Reduce Parsing for the given input strings using the grammar

$S \rightarrow (L)|a$

$L \rightarrow L,S|S$

i) $(a,(a,a))$

ii) (a,a)

- *Syntax Analyzer*
- creates the syntactic structure of the given source program.
- This syntactic structure is mostly a *parse tree*
- .Syntax Analyzer is also known as *parser*
- LR parsers can be constructed to recognize virtually all programming-language constructs for which context-free grammars can be written.
- Bottom-up parsers for a large class of context-free grammars can be easily developed using operator grammars. Operator grammars have the property that no production right side is empty or has two adjacent nonterminals.
- A shift-reduce parser uses a parse stack which (conceptually) contains grammar symbols. During the operation of the parser, symbols from the input are shifted onto the stack. If a prefix of the symbols on top of the stack matches the RHS of a grammar rule which is the correct rule to use within the current context, then the parser reduces the RHS of the rule to its LHS, replacing the RHS symbols on top of the stack with the nonterminal occurring on the LHS of the rule.
- This shift-reduce process continues until the parser terminates, reporting either success or failure. It terminates with success when the input is legal and is accepted by the parser.

1. K. Muneeswaran, Compiler Design, First Edition, Oxford University Press.
2. J.P. Bennet, “Introduction to Compiler Techniques”, Second Edition, Tata McGraw-Hill, 2003.
3. Henk Alblas and Albert Nymeyer, “Practice and Principles of Compiler Building with C”, PHI, 2001.
4. Aho, Sethi & Ullman, "Compilers: Principles, Techniques and Tools", Pearson Education
5. V Raghvan, “ Principles of Compiler Design”, TMH
6. Kenneth Loudon,” Compiler Construction”, Cengage Learning.
7. Charles Fischer and Ricard LeBlanc,” Crafting a Compiler with C”, Pearson Education



Thank You